MetaGPT: A Multi-Agent Framework with RAG for Code Generation

CSD416 PROJECT PHASE II

Project Phase II Report submitted in partial fulfillment for the degree of
B.Tech Computer Science and Engineering

Submitted by

ADARSH A S (Reg No: MUT22CS005)
ANUPAM KRISHNA T V (Reg No: MUT22CS032)
ASLAH T P (Reg No: MUT22CS041)
KARTHIKA NAIR (Reg No: MUT22CS081)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(Affiliated to APJ Abdul Kalam Technological University)

MARCH 2026

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously written by another person nor material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgement has been made in the text.

ADARSH A S (Reg No: MUT22CS005)
ANUPAM KRISHNA T V (Reg No: MUT22CS032)
ASLAH T P (Reg No: MUT22CS041)
KARTHIKA NAIR (Reg No: MUT22CS081)

Place:

Date:

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Project Phase II report entitled **"MetaGPT: A Multi-Agent Framework with RAG for Code Generation"**, *submitted by* **ADARSH A S, ANUPAM KRISHNA T V, ASLAH T P, and KARTHIKA NAIR** *of Semester VIII is a bonafide account of the work done by them under our supervision during the academic year 2025-26.*

Guide:	**Coordinator:**	**Head of the Department:**
Dr. Abdul Ali	Dr. Sheena Kurian K	Dr. Anishin Raj M M
Assistant Professor	*Assistant Professor*	*Head of the Department*
Dept. of CSE	Dept. of CSE	Dept. of CSE

Acknowledgements

This project report is the result of our hard work wherein we have been helped and supported by several persons and institutions directly and indirectly. Now it is the time to acknowledge their valuable contributions.

We would like to extend our sincere gratitude to the **Management of Muthoot Institute of Technology and Science** for providing us with the necessary facilities, resources, and a conducive environment to carry out our project successfully. We are deeply thankful to **Dr. Neelakantan P. C.**, Principal, Muthoot Institute of Technology and Science, for his constant support, guidance, and encouragement throughout our academic journey. Our sincere gratitude to **Dr.Anishin Raj M M**, Head of the Department during our course period, for his guidance and support. His encouragement and leadership have greatly helped us in successfully completing our project.

We would like to express our heartfelt gratitude to our project guide, **Guide name** , for his/her continuous encouragement, invaluable guidance, motivation, and enthusiasm throughout our project.. We would also like to thank the project coordinators **Asst.Prof. Dr.Sheena Kurian K** and **Asst.Prof. Mithu Mary George**, for critically assessing our work and providing valuable suggestions.

In our daily work, we have been blessed with a friendly and cheerful group of fellow students. We are very thankful to all the members of S8 CSE.

Besides this, several people have knowingly and unknowingly helped us in the successful completion of this project. We express our sincere gratitude to all of them.

Abstract

Modern software teams increasingly turn to large language models (LLMs) to accelerate development, yet many tools still generate code in an ad-hoc, one-shot fashion with weak structure, limited verification, and poor support for iterative refinement. This project, **MetaGPT**, implements an end-to-end agentic framework that transforms natural language prompts into complete software projects using a coordinated pipeline of specialized LLM-driven agents. The system enables users to create projects, inspect intermediate artifacts, and iteratively refine outputs through an interactive interface.

MetaGPT models a software team through four core agents—Manager, Architect, Engineer, and QA—each governed by explicit Standard Operating Procedures (SOPs). The Manager converts informal prompts into structured requirements, the Architect designs system architecture and file layouts, the Engineer generates implementation code, and the QA agent derives tests and quality feedback. These agents are executed as a stateful workflow with preserved intermediate results, enabling traceability from requirements to design to implementation. The system persists project state and generated artifacts to support repeatability, recovery, and long-running iterations, and it incorporates retrieval utilities to ground future improvements in existing project context. Evaluation on representative software prompts shows that this SOP-driven, workflow-orchestrated approach yields more consistent project structures, improved observability of agent outputs, and better support for iterative refinement compared to naive, single-step code generation, demonstrating MetaGPT's viability as a practical foundation for agentic software engineering.

Contents

Abstract **i**

List of Figures **iv**

List of Tables **v**

List of Abbreviations **vi**

1 Introduction **1**
1.1 Overview . . . 1
1.2 Background . . . 1
1.3 Motivation . . . 2

2 Literature Review **3**
2.1 Introduction . . . 3
2.2 Key Papers Reviewed . . . 3
2.2.1 MetaGPT: Meta Programming for Multi-Agent Collaborative Framework (Hong et al., 2023) . . . 4
2.2.2 Evaluating Large Language Models Trained on Code (Codex, OpenAI 2021) 5
2.2.3 Competition-Level Code Generation with AlphaCode (DeepMind, 2022) . 5
2.2.4 Auto-GPT for Online Decision Making (2023) . . . 6
2.2.5 MapCoder: Multi-Agent Code Generation for Competitive Problem Solving (2024) . . . 6
2.3 Literature Matrix . . . 7
2.4 MetaGPT Framework . . . 10
2.5 Research Gaps and Our Contribution . . . 11

3 Problem Identification And Objectives **12**
3.1 Problem Statement . . . 12
3.2 Objectives . . . 12
3.3 Scope and Applications . . . 13
3.4 Feasibility Analysis . . . 13
3.5 Project Plan . . . 14

4 Methodology **17**
4.1 High Level Design . . . 17
4.1.1 Block Diagram . . . 17

4.1.2 Use Case Diagram . . . 18
4.2 Detailed Design . . . 18
4.2.1 Module 1: Backend API Layer . . . 18
4.2.2 Module 2: Agent Orchestration and Graph . . . 19
4.2.3 Module 3: Storage and Project Management . . . 20
4.2.4 Module 4: Frontend User Interface . . . 20
4.2.5 Module 5: LLM, SOP, and Retrieval . . . 21
4.3 Data Flow Diagram . . . 21
4.3.1 Context Diagram (Level 0 DFD) . . . 21
4.3.2 Level 1 DFD . . . 22
4.3.3 Level 2 DFD . . . 23
4.4 ER Diagram . . . 24
4.5 UI Design . . . 24

5 Implementation And Results **33**
5.1 Software And Hardware Requirements . . . 33
5.2 Implementation Details . . . 33
5.3 Testing . . . 42
5.3.1 Unit Testing . . . 42
5.3.2 Integration Testing . . . 44
5.3.3 System Testing . . . 49
5.4 Experimental Results & Discussion . . . 50

6 Conclusion **52**

7 Future Scope **53**

References **54**

Appendix **56**

List of Figures

4.1 High-Level MetaGPT System Architecture 17
4.2 MetaGPT System Use Case Diagram 18
4.3 DFD Level 0 - MetaGPT System 22
4.4 DFD Level 1 - MetaGPT System Internal Workflow 23
4.5 DFD Level 2 — Detailed Agent Collaboration Process 23
4.6 MetaGPT — Home Page 24
4.7 MetaGPT — Home Page with Prompt Entered 25
4.8 MetaGPT — Sign In Page 25
4.9 MetaGPT — Sign Up Page 26
4.10 MetaGPT — List of Projects 26
4.11 MetaGPT — Manager Agent Output 27
4.12 MetaGPT — Architect Agent Output 28
4.13 MetaGPT — Engineer Agent Output 29
4.14 MetaGPT — QA Agent Output 30
4.15 MetaGPT — QA Agent Output (continued) 31
4.16 MetaGPT — Code Preview 32
4.17 MetaGPT — Sandbox Live Preview 32

List of Tables

2.1 Literature Matrix: Comparative Analysis of Key Papers 8
2.2 Literature Matrix: Comparative Analysis of Key Papers (Continued) 9

5.1 Unit Test Cases for MetaGPT Modules 42
5.2 Integration Test Cases: API-level Tests 45
5.3 Integration Test Cases: Pipeline Orchestration 46
5.4 Integration Test Cases: Storage + API Integration 46
5.5 Integration Test Cases: RAG Workflows 47
5.6 Integration Test Cases: Backend to Frontend 48
5.7 System Test Cases: Project as a Whole 49

List of Abbreviations

1. API — Application Programming Interface
2. CI — Continuous Integration
3. CORS — Cross-Origin Resource Sharing
4. CRUD — Create, Read, Update, Delete
5. CSS — Cascading Style Sheets
6. DFD — Data Flow Diagram
7. E2B — Environment-to-Browser (sandboxed preview environment)
8. ER — Entity-Relationship
9. HTTP — Hypertext Transfer Protocol
10. IDE — Integrated Development Environment
11. JSON — JavaScript Object Notation
12. LLM — Large Language Model
13. NLP — Natural Language Processing
14. ODM — Object Document Mapper
15. QA — Quality Assurance
16. RAG — Retrieval-Augmented Generation
17. REST — Representational State Transfer
18. SOP — Standard Operating Procedure
19. SRS — Software Requirements Specification
20. SSE — Server-Sent Events
21. UI — User Interface

Chapter 1

Introduction

1.1 Overview

Recent advances in large language models (LLMs) have enabled the automatic generation of high-quality source code from natural language requirements. However, most existing tools operate as single-step assistants that produce code snippets without a systematic process for requirements analysis, architecture design, implementation, testing, and iteration. This often leads to code that is difficult to maintain, lacks structure, and does not align with production engineering practices.

MetaGPT is a production-ready agentic code generation system designed to address these limitations. It uses a pipeline of specialized autonomous agents, each governed by a Standard Operating Procedure (SOP), to transform user prompts into complete, structured software projects. The system exposes a FastAPI-based backend for orchestration and a Next.js frontend for interactive project management, file exploration, and chat-based refinement. By combining SOP-driven agents with modern web technologies, MetaGPT aims to make LLM-assisted development more reliable, repeatable, and suitable for real-world use.

1.2 Background

Traditional software development workflows follow a sequence of phases such as requirement gathering, analysis, design, implementation, testing, and maintenance. In practice, these phases are iterative and collaborative, involving multiple roles like project managers, architects, developers, and quality assurance (QA) engineers. With the emergence of LLMs, there has been a shift towards automating parts of this workflow, particularly in code synthesis and documentation generation.

Despite this progress, several challenges remain. One-shot code generation often lacks traceability from requirements to implementation. Generated projects may not follow consistent architectures, naming conventions, or testing strategies, and they rarely integrate into existing continuous integration or deployment pipelines. Furthermore, many tools depend on

a single monolithic prompt, making it difficult to adapt or extend the system for new project types.

MetaGPT is built to bridge this gap by explicitly modeling the roles and responsibilities of different agents in a software team. Each agent in the pipeline—Manager, Architect, Engineer, and QA—is implemented as a separate component that consumes and produces structured data. This design promotes modularity, extensibility, and transparency in how LLMs are used to build software projects.

1.3 Motivation

Developers and product teams increasingly seek tools that can accelerate the initial stages of software creation while preserving code quality and architectural integrity. Manually translating informal prompts into full-stack applications is time-consuming and prone to misinterpretation, especially when iterating on requirements. At the same time, organizations require systems that are transparent, auditable, and easy to integrate into existing engineering workflows.

MetaGPT is motivated by the need for an end-to-end agentic platform that can reliably generate, organize, and refine codebases based on user prompts. By formalizing agent behavior through SOPs and exposing clear APIs for project creation, pipeline execution, and file management, the system aims to reduce manual effort while preserving control and oversight. The project also serves as a reference implementation for building LLM-powered developer tools that emphasize structure, observability, and extensibility over ad-hoc code generation.

Chapter 2

Literature Review

2.1 Introduction

The field of multi-agent systems and automated code generation has evolved significantly with the advent of Large Language Models (LLMs). This literature review examines the current state of the art in multi-agent frameworks, code generation techniques, and the challenges that our MetaGPT project aims to address.

Recent research has demonstrated remarkable progress in automated problem solving through societies of agents based on large language models. However, existing LLM-based multi-agent systems can only solve simple dialogue tasks effectively. Solutions to more complex tasks are complicated through logic inconsistencies due to cascading hallucinations caused by naively chaining LLMs.

The research landscape reveals several critical challenges that our project addresses:

- **Logic Inconsistencies:** Existing systems suffer from cascading hallucinations when chaining LLMs, leading to unreliable outputs.
- **Lack of Structured Workflows:** Current multi-agent frameworks lack standardized operating procedures that guide agent interactions.
- **Communication Inefficiencies:** Agents often communicate through unconstrained natural language, leading to information distortion and inefficient collaboration.
- **Insufficient Self-Correction:** Most systems lack robust mechanisms for debugging and improving generated code.

Our MetaGPT framework directly addresses these challenges by incorporating Standardized Operating Procedures (SOPs) into multi-agent collaborations, enabling agents with human-like domain expertise to verify intermediate results and reduce errors.

2.2 Key Papers Reviewed

The literature on multi-agent systems and automated code generation provides a rich foundation for understanding the challenges and opportunities in this domain. Several key research papers

have shaped the field of automated code generation and multi-agent systems, each addressing different aspects of the problem space.

2.2.1 MetaGPT: Meta Programming for Multi-Agent Collaborative Framework (Hong et al., 2023)

Authors: H. Hong, Y. Du, J. Pan, Z. Wu, and W. Chen

Problem Addressed: How can multiple LLM agents collaborate to solve complex software engineering tasks with reduced hallucination and higher consistency compared to naive multi-agent chat?

Core Idea and Methodology:

- Proposes a "meta programming" view of multi-agent collaboration, where agents follow structured roles and produce explicit intermediate artifacts (e.g., requirement documents, design plans, and code deliverables) rather than relying only on free-form dialogue.
- Emphasizes **role specialization** (e.g., product-oriented analysis, architecture design, engineering implementation, and quality assurance) to mimic real software teams.
- Introduces **standardized workflows** to reduce information loss across agent handoffs and to make the process more auditable and repeatable.

Strengths:

- Encourages structured communication through artifacts, improving traceability from requirements to implementation.
- Provides a clear decomposition of responsibilities across agents, making it easier to scale to multi-step software tasks.
- Highlights the importance of verification and quality checks as part of the workflow rather than an afterthought.

Weaknesses / Limitations:

- Practical deployments still face engineering challenges such as persistence, observability, and tool/runtime integration for executing and validating generated projects.
- Performance and reliability depend on the quality of agent prompts, structured outputs, and the availability of mechanisms to recover from failures.

Relevance to our project: The proposed framework adopts this central idea by implementing a role-based pipeline (Manager, Architect, Engineer, QA) with explicit SOP definitions, graph-based orchestration, persistent storage of project state, and support services (e.g., sandboxed previews) to operationalize the research concepts into an end-to-end system.

2.2.2 Evaluating Large Language Models Trained on Code (Codex, OpenAI 2021)

Authors: Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan

Problem Addressed: Can large language models (trained on natural language) be adapted for code generation?

Methodology:

- Fine-tuned GPT-3 on a massive GitHub code corpus to create Codex
- Generates code from natural language prompts and docstrings
- Benchmarked on HumanEval programming problems

Strengths:

- Fluent code generation capabilities
- Effective for autocomplete, boilerplate, and simple scripts
- Demonstrated the potential of LLMs for code generation

Weaknesses:

- Struggles with multi-file and system-level projects
- Prone to hallucination and insecure code generation
- Limited verification beyond sampling and reranking

2.2.3 Competition-Level Code Generation with AlphaCode (DeepMind, 2022)

Authors: Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser

Problem Addressed: Can LLMs solve competitive programming problems that require advanced reasoning, algorithm design, and coding?

Methodology:

- Trained transformer-based models on competitive programming datasets
- Generated a very large pool of candidate programs through sampling
- Applied clustering and filtering to remove near-duplicates and select diverse solutions
- Used unit testing to automatically execute programs on hidden test cases

Strengths:

- Strong performance on algorithmic challenges
- Effective verification via unit tests
- Robust solution space exploration through massive sampling

Weaknesses:

- Extremely compute-intensive (generates hundreds of thousands of candidates)
- Focused on isolated problems, not full software projects
- Lacked structured planning or collaboration mechanisms

2.2.4 Auto-GPT for Online Decision Making (2023)

Authors: Hui Yang, Sifu Yue, Yunzhong He

Problem Addressed: Can LLMs operate autonomously, making decisions and using tools to complete multi-step goals without human step-by-step input?

Methodology:

- Introduced Auto-GPT as an experimental open-source framework
- Uses chains of LLM prompts with a looping mechanism:
  - Plan next step
  - Execute (tool use, web browsing, file editing)
  - Evaluate outcome
  - Continue until goal achieved
- Applied mainly to online decision-making tasks (web search, file operations, basic software tasks)

Strengths:

- Handles long-horizon tasks beyond one-shot prompting
- Flexible and can adapt to new tasks via tool use
- Pioneered the autonomous LLM trend

Weaknesses:

- Brittle and prone to getting stuck in loops
- No structured planning (SOPs) leading to ad-hoc decision-making
- Weak verification produces errors without correction

2.2.5 MapCoder: Multi-Agent Code Generation for Competitive Problem Solving (2024)

Authors: Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser

Problem Addressed: Can multi-agent LLMs collaborate to solve competitive programming problems efficiently?

Methodology:

- Each agent specializes in a subtask:
  - Planner → decomposes problem into smaller steps
  - Solver agents → generate candidate code fragments
  - Verifier → runs tests and identifies errors
- Agents communicate iteratively, exchanging partial solutions
- Evaluated on competitive programming datasets (Codeforces, AtCoder)

Strengths:

- Efficient multi-agent coordination reduces computational cost compared to AlphaCode's massive sampling
- Verification is embedded in the workflow
- Parallel agent execution allows scalability

Weaknesses:

- Focused on competitive programming, not full software projects
- Limited grounding as agents rely mainly on LLM reasoning, not external documentation

2.3 Literature Matrix

Table 2.1 provides a comprehensive comparison of the key papers reviewed, highlighting their methodologies, strengths, weaknesses, and relevance to our MetaGPT framework.

Table 2.1: Literature Matrix: Comparative Analysis of Key Papers

Sl.No	TITLE	AUTHOR(S)	METHODOLOGY	DATASET/CONTEXT	ADVANTAGES	DISADVANTAGES
1	Chain-of-Thought Prompting Elicits Reasoning in Large Language Models	Jason Wei et al. (2023)	Introduces chain-of-thought (CoT) prompting with intermediate reasoning steps	Arithmetic reasoning: GSM8K, SVAMP, ASDiv, AQUA, MAWPS; Commonsense reasoning: CSQA, StrategyQA, SayCan, BIG-Bench tasks	Enables multi-step reasoning in LMs (improves arithmetic, commonsense, symbolic tasks)	Only works well with very large models, High computational cost for inference with huge models
2	ReAct: Synergizing Reasoning and Acting in Language Models	Shunyu Yao et al. (2023)	Introduces ReAct prompting: interleaves reasoning traces with actions	ALFWorld (text-based games), WebShop (online shopping with user instructions)	Combines internal reasoning with external actions, reducing hallucinations	More reasoning errors than CoT due to rigid structure (sometimes loops or repeats)
3	CodeTree: Agent-guided Tree Search for Code Generation with Large Language Models	Jierui Li et al. (2025)	Framework for LLM agents using unified tree structure to explore search space for code generation	Function implementation tasks (HumanEval, MBPP, HumanEval+, MBPP+) and Program implementation tasks (CodeContests, APPS)	Flexible and efficient approach that avoids redundant exploration of solutions	Struggles with very difficult, competition-level problems
4	Exploration of LLM Multi-Agent Application Implementation Based on LangGraph+Crew AI	Zhihua Duan and Jialin Wang (2024)	Use of LangGraph and CrewAI in multi agent system for code generation	Demonstrates framework capabilities through application cases in code generation, code review, ticket auditing and forwarding	Optimizes task execution and enhances system flexibility and scalability through real-time status data sharing and feedback mechanisms	The paper does not explicitly state any disadvantages of using the LangGraph and CrewAI frameworks together
5	Enhancing Function-Calling Capabilities in LLMs: Strategies for Prompt Formats, Data Integration, and Multilingual Translation	Yi-Chang Chen et al. (2024)	Tuning-based approach to improve function-calling in Large Language Models	110k instruction-following instances from Open ORCA (IF-110k) and 110k function-calling instances (FC-110k) from APIGen	Instruction-following data significantly improves function-calling accuracy and relevance detection	The Decision Token approach resulted in a slight decrease in function-calling accuracy
6	Large Language Model based Multi-Agents: A Survey of Progress and Challenges	Taicheng Guo et al. (2024)	Summarizes commonly used datasets, benchmarks, and open-source frameworks	N/A	Highlights the advantages of LLM-based multi-agent systems over single-agent systems	Identifies the escalating complexity of managing a large number of agents as a critical problem

Table 2.2: Literature Matrix: Comparative Analysis of Key Papers (Continued)

Sl.No	TITLE	AUTHOR(S)	METHODOLOGY	DATASET/CONTEXT	ADVANTAGES	DISADVANTAGES
7	Agent AI with LangGraph: A Modular Framework for Enhancing Machine Translation Using Large Language Models	Jialin Wang and Zhihua Duan (2024)	Intent Agent to analyze input text and route to appropriate translation agent using LLMs	Uses English and French sentence pairs to train and test the model. A total of 75,000 words were selected for the experiment	High degree of flexibility, scalability, and accuracy in multilingual translation tasks	The model structure was relatively simple and the number of data sets was insufficient
8	Retrieval Augmented Code Generation and Summarization	Md Rizwan Parvez et al. (2021)	Proposes two-step, retrieval-augmented framework called REDCODER with retriever and generator modules	CodeXGLUE and Concode benchmarks, framework was evaluated on code generation and summarization tasks in Java and Python	Enhances code generation and summarization by augmenting input with retrieved information	Computational expense of cross-encoder code retrievers and inability of sparse token-level retrievers (like BM25) to find semantically similar code and summaries
9	Evaluating Large Language Models Trained on Code (Codex)	Mark Chen, Jerry Tworek et al. (2021)	Fine-tuned GPT-3 on a massive GitHub code corpus; benchmarked on HumanEval programming problems	HumanEval (164 hand-written programming problems)	Demonstrated the potential of LLMs for code generation; effective for autocomplete, boilerplate, and simple scripts	Struggles with multi-file and system-level projects; prone to hallucination and insecure code generation
10	Competition-Level Code Generation with AlphaCode	Yujia Li, David Choi et al. (2022)	Trained transformer-based models on competitive programming datasets; generated large pool of candidates via sampling, then filtered using clustering and unit tests	CodeContests competitive programming dataset	Strong performance on algorithmic challenges; effective verification via unit tests; robust solution space exploration	Extremely compute-intensive; focused on isolated problems, not full software projects; no structured planning or collaboration
11	Auto-GPT for Online Decision Making	Hui Yang, Sifu Yue, Yunzhong He (2023)	Chains LLM prompts in a plan-execute-evaluate loop with tool use (web browsing, file editing)	Online decision-making tasks including web search and file operations	Handles long-horizon tasks; flexible tool use; pioneered the autonomous LLM agent trend	Brittle and prone to getting stuck in loops; no structured SOPs; weak verification produces errors without correction
12	MapCoder: Multi-Agent Code Generation for Competitive Problem Solving	Md. Ashraful Islam et al. (2024)	Multi-agent pipeline: Planner decomposes problem, Solver agents generate code, Verifier runs tests and feeds back iteratively	Competitive programming datasets: Codeforces, AtCoder, HumanEval, MBPP, APPS	Efficient multi-agent coordination; verification embedded in workflow; parallel execution allows scalability	Focused on competitive programming, not full software projects; limited grounding in external documentation
13	MetaGPT: Meta Programming for Multi-Agent Collaborative Framework	H. Hong, Y. Du, J. Pan, Z. Wu, and W. Chen (2023)	Meta programming view: agents follow structured roles and produce explicit intermediate artifacts (requirements, design plans, code) rather than free-form dialogue	HumanEval, MBPP, and SoftwareDev software engineering benchmarks	Structured communication improves traceability; clear decomposition of responsibilities; verification and quality checks are part of the workflow	Practical deployments face challenges in persistence and observability; performance depends heavily on prompt quality and structured output reliability

The literature matrix reveals several key insights:

- **Evolution of Approaches:** From single LLM (Codex) to multi-agent systems (MapCoder), showing progression toward collaborative frameworks.
- **Verification Gaps:** Most approaches lack robust verification mechanisms, with only AlphaCode implementing comprehensive testing.
- **Structured Workflow Deficiency:** None of the existing approaches implement structured workflows or Standardized Operating Procedures (SOPs).
- **Context Awareness Limitations:** All approaches rely primarily on LLM reasoning without external knowledge integration.
- **Scalability Challenges:** Most approaches face scalability issues, either through computational intensity or lack of structured planning.

2.4 MetaGPT Framework

To address the limitations of existing multi-agent systems, particularly the logic inconsistencies and cascading hallucinations caused by naively chaining LLMs, we introduce the MetaGPT framework. Based on the foundational concepts proposed by Hong et al. (2023) in their paper "MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework", MetaGPT incorporates efficient human workflows into LLM-based multi-agent collaborations through several key innovations:

- **Standardized Operating Procedures (SOPs):** MetaGPT encodes human domain expertise and established SOPs into prompt sequences. This streamlines workflows and significantly improves the consistency and accuracy of task execution compared to free-form dialogue.
- **Role Specialization and Complete Workforce:** The framework utilizes an assembly line paradigm, assigning specialized roles such as Product Manager, Architect, Project Manager, Engineer, and QA Engineer. Each role follows specific constraints and responsibilities, mimicking a real-world software company to efficiently break down complex tasks.
- **Structured Communication Protocols:** Instead of unconstrained natural language chatter, agents communicate through structured artifacts like Product Requirement Documents (PRDs), system designs, APIs, and flowcharts. This minimizes ambiguities, reduces information distortion, and prevents the "telephone game" effect.
- **Publish-Subscribe Message Mechanism:** Agents share information through a global message pool. Rather than one-to-one communication, agents publish their structured outputs to the pool and subscribe to relevant messages based on their role profiles, avoiding information overload and improving efficiency.

- **Executable Feedback and Iterative Programming:** To ensure code quality and executability, MetaGPT employs a self-correction mechanism during runtime. Engineers generate code, write unit tests, and execute them. If errors occur, the agent debugs the code using historical execution memory and past artifacts, improving the code generation iteratively.

Extensive evaluations on collaborative software engineering benchmarks, such as HumanEval and MBPP, demonstrate that MetaGPT achieves state-of-the-art performance, validating the effectiveness of structured workflows and specialized multi-agent collaboration in complex automated programming tasks.

2.5 Research Gaps and Our Contribution

Grounded in the surveyed literature and current tool landscape, several concrete gaps can be identified that are directly relevant to our proposed project:

- **Lack of end-to-end, production-oriented systems:** Many works focus either on solving isolated programming problems (e.g., competitive programming) or on early research prototypes of multi-agent systems. There is a shortage of production-ready platforms that take a natural language prompt all the way to a structured, full-stack project with APIs, UI, tests, and persistent storage.

- **Weak role modeling and orchestration:** Existing multi-agent approaches often describe roles conceptually, but do not provide a concrete, reusable orchestration layer where specialized agents are wired together with explicit state and transitions. Robust graph-based orchestration for software projects remains underexplored.

- **Limited observability and traceability:** Most systems do not expose detailed reasoning traces, intermediate artifacts, or execution timelines to end users. This makes it difficult for developers to understand why a system produced a particular codebase or to debug agent behavior.

- **Insufficient integration with modern web development stacks:** Few research systems are tightly integrated with contemporary stacks such as FastAPI on the backend and Next.js/React on the frontend, with opinionated project structures that align with how teams actually build and ship applications.

- **Underutilization of retrieval for code-level context:** While retrieval-augmented generation has been explored for natural language and small code snippets, there is limited work on integrating RAG deeply into agentic workflows for managing entire projects, file trees, and prior artifacts.

These gaps motivate the design choices in our framework, which explicitly targets end-to-end, observable, and stack-aware agentic code generation.

Chapter 3

Problem Identification And Objectives

3.1 Problem Statement

Although large language models are capable of generating high-quality code snippets, most current tools treat code generation as a single-step transformation from a prompt to source files. This approach lacks a systematic process for requirement analysis, architecture design, iterative refinement, and validation, leading to inconsistent project structures and limited support for maintenance. There is no explicit notion of roles, responsibilities, or quality gates, and generated projects often fail to meet production standards in terms of testing, modularity, and documentation.

The problem addressed in this project is to design and implement an **agentic code generation system** that can transform natural language specifications into complete, production-ready codebases using a structured, multi-agent pipeline. The system should coordinate specialized agents that emulate the roles of a human software team and should expose clear APIs and user interfaces to create, inspect, and iteratively improve projects generated by the pipeline.

3.2 Objectives

The primary objective of this project is to develop **MetaGPT**, a production-ready, SOP-driven agentic system for code generation. The specific objectives are:

- To design a multi-agent pipeline consisting of Manager, Architect, Engineer, and QA agents that operate on structured inputs and outputs.
- To implement a FastAPI backend that exposes RESTful endpoints for project creation, pipeline execution, file management, and chat-based iteration.
- To integrate Google Gemini via LangChain and LangGraph for orchestrating the agent workflow and maintaining pipeline state.
- To develop a Next.js frontend that provides a dark-themed user interface for viewing agent outputs, browsing generated files, monitoring pipeline execution, and interacting with

projects.

- To provide an isolated **live preview** capability for generated projects using an external sandbox service with non-blocking creation and status polling.
- To persist project metadata, pipeline status, and agent outputs in a **MongoDB** database to enable recovery, multi-user access control, and consistent iteration over time.
- To evaluate the system in terms of usability, consistency of generated project structure, and support for iterative refinement compared to one-shot LLM-based tools.

3.3 Scope and Applications

The scope of MetaGPT primarily targets full-stack web application generation, where the system can detect and scaffold common frameworks such as React and Next.js on the frontend and FastAPI-based services on the backend. The architecture, however, is intentionally generic and can be extended to support other project templates, languages, and deployment targets by updating the agent SOPs and project templates.

In the current implementation, the scope includes operational features required for practical usage: persistent storage of project state in MongoDB and optional live preview of generated projects through an isolated sandbox environment. These capabilities support longer workflows, interactive debugging, and reliable project access across application restarts.

The system can be applied in several scenarios:

- Rapid prototyping of new product ideas by generating initial codebases from high-level prompts.
- Educational settings, where students can observe how requirements are translated into architecture, implementation, and tests by different agents.
- Internal tooling for organizations that wish to standardize how LLMs are used to generate and refactor code according to established conventions.
- As a foundation for future research on agentic software engineering, including automatic refactoring, performance optimization, and continuous integration with human feedback.

3.4 Feasibility Analysis

The feasibility of MetaGPT can be analyzed across technical, economic, and operational dimensions:

- **Technical Feasibility**: The backend is implemented in Python 3.11 using FastAPI, which is well-suited for building high-performance, asynchronous APIs. Integration with Google Gemini is achieved through LangChain and LangGraph, both of which provide mature abstractions for LLM orchestration and agent workflows. The frontend is built with Next.js 14, TypeScript, and Tailwind CSS, technologies that are widely adopted and supported by the developer community.

- **Economic Feasibility**: The system is designed to run on commodity hardware with no requirements for proprietary infrastructure beyond the LLM API key. The use of open-source frameworks (FastAPI, Next.js, Tailwind CSS, and others) reduces licensing costs. Depending on usage patterns, LLM inference costs can be controlled by batching requests, caching intermediate artifacts, and tuning model parameters.
- **Operational Feasibility**: The project structure separates backend and frontend concerns, making it straightforward to deploy and maintain the system. Clear environment configuration files, documentation, and scripts for testing, linting, and type checking support day-to-day operations. The RESTful API and web interface make the system accessible to both developers and non-technical stakeholders.

3.5 Project Plan

The project was executed using a phased, iterative plan that covered analysis, design, implementation, testing, and documentation. Each phase had clearly defined deliverables, timelines, and responsibilities.

Phase I – Requirement Analysis and High-Level Design (Week 1–2)

- Conducted a survey of existing LLM-based code generation tools and agentic frameworks.
- Refined the problem statement, objectives, scope, and feasibility based on literature review and initial experiments.
- Identified primary user personas (developer, student, researcher) and their interaction flows with the system.
- Drafted the high-level architecture, including the separation into backend, agent pipeline, storage, and frontend subsystems.
- Defined initial success criteria (e.g., ability to generate a multi-file React/FastAPI project from a single prompt).

Phase II – Backend and Agent Pipeline Design (Week 3–4)

- Designed the project structure under `backend/app` including `api`, `agents`, `graph`, `services`, `storage`, `schemas`, and `rag`.
- Specified the responsibilities and data contracts for the Manager, Architect, Engineer, and QA agents.
- Defined the LangGraph pipeline state in `graph/state.py` and documented the transitions in `graph/orchestrator.py`.
- Designed REST API endpoints for projects, pipeline execution, files, chat, and RAG operations in the `api/endpoints` package.
- Finalized environment configuration parameters in `config.py` and designed .env templates.

Phase III – Backend Implementation (Week 5–7)

- Implemented the FastAPI application entry point in `main.py` and wired the routers defined in `api/router.py`.
- Implemented agent classes in `agents/manager.py`, `agents/architect.py`, `agents/engineer.py`, and `agents/qa.py`, following SOPs from `sop/definitions.py`.
- Developed pipeline and chat services in `services/pipeline_service.py` and `services/chat_service.py`.
- Implemented storage backends for projects and files in `storage/project_store.py` and `storage/file_store.py`.
- Integrated Google Gemini configuration via LangChain in `llm/gemini.py`, including helper functions for structured output.

Phase IV – Frontend Implementation (Week 8–10)

- Created the Next.js App Router structure under `frontend/src/app`, including landing, projects list, project workspace, and authentication pages.
- Implemented core UI components such as `AgentOutputs`, `ChatPanel`, `CodeViewer`, `ExecutionTimeline`, and `FileExplorer`.
- Set up global styling and theming with Tailwind CSS and dark theme defaults.
- Implemented API client utilities in `frontend/src/lib/api.ts` and application state management using stores in `frontend/src/lib/store.ts`.
- Connected frontend interactions (project creation, pipeline run, file viewing, chat) to backend endpoints.

Phase V – Testing, Evaluation, and Refinement (Week 11–12)

- Wrote and executed unit tests for backend services, schemas, and utility functions using `pytest`.
- Performed integration tests covering project lifecycle, pipeline execution, and file retrieval operations.
- Ran linting and type-checking for both backend and frontend (`ruff`, `mypy`, ESLint, TypeScript).
- Evaluated the system using representative prompts and collected observations about code quality, structure, and usability.
- Refined agent prompts, SOP definitions, and UI feedback based on test results and manual inspection.

Phase VI – Documentation and Deployment Readiness (Week 13)

- Prepared project documentation including top-level `README.md`, backend and frontend READMEs, and this report.
- Documented environment setup steps, configuration options, and development workflows.
- Verified that the project can be cloned, configured, and run from a clean environment using the documented instructions.
- Identified future enhancements and roadmap items for subsequent iterations.

Chapter 4

Methodology

4.1 High Level Design

4.1.1 Block Diagram

The MetaGPT framework follows a sophisticated multi-tier architecture that supports distributed agent interactions, intelligent communication protocols, and RAG-enhanced knowledge processing. The system is organized into clearly defined layers that collaborate to transform user requirements into validated code artifacts.

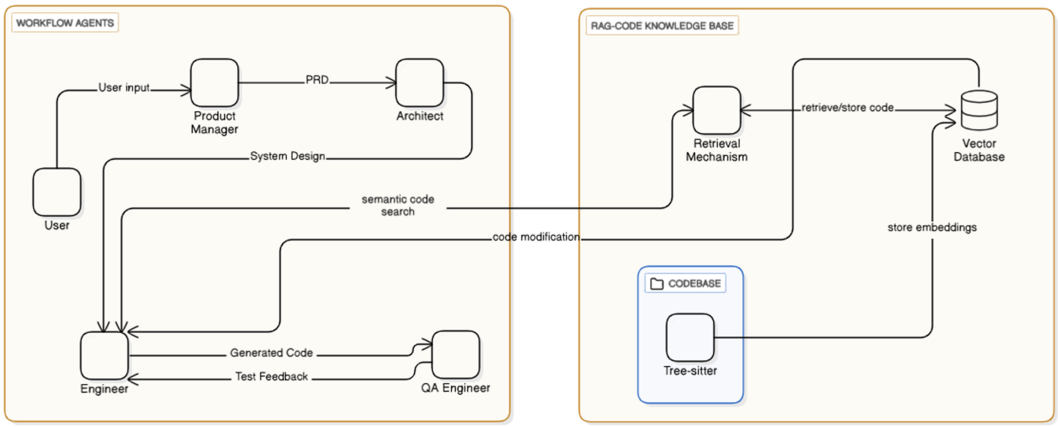

Figure 4.1: High-Level MetaGPT System Architecture

The architecture diagram in Figure 4.1 illustrates the following key components:

- **Frontend Layer:** React-based user interface providing agent management, workflow visualization, and real-time collaboration monitoring.
- **API Gateway:** FastAPI backend serving as the central orchestration point for all agent interactions and workflow management.
- **Agent Framework:** LangGraph-based state graph providing deterministic agent orchestration, with LangChain used as the LLM abstraction layer for calling Google Gemini.

- **Specialized Agents:** Manager, Architect, Engineer, and QA agents with distinct roles and responsibilities.
- **RAG System:** Vector database integration with semantic search capabilities for context-aware code generation and knowledge retrieval.
- **AI Integration:** Gemini API integration providing state-of-the-art language model capabilities for agent reasoning and code generation.
- **Communication Layer:** Shared-state passing via a pipeline service, where each agent reads typed outputs from the previous agent and writes its own output into the same state object.

4.1.2 Use Case Diagram

The Use Case diagram illustrates the interactions between the primary actors (User and Multi-Agent System) and the MetaGPT framework, showing the complete software development lifecycle from user input to final code delivery.

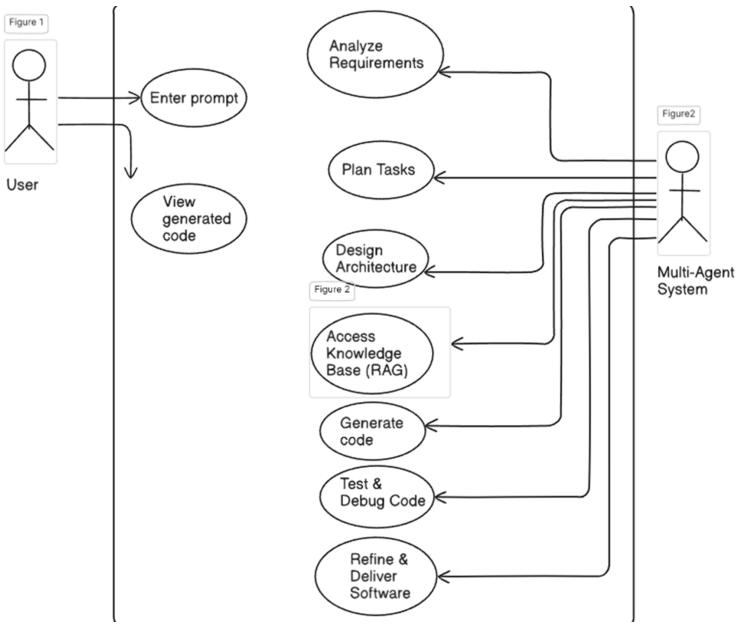

Figure 4.2: MetaGPT System Use Case Diagram

4.2 Detailed Design

4.2.1 Module 1: Backend API Layer

The first major module is the **Backend API Layer**, implemented using FastAPI under the `backend/app` package. Its primary responsibility is to expose a stable HTTP interface for all core operations, including project management, pipeline execution, file access, chat-based iteration, and retrieval-augmented generation.

The entry point `main.py` constructs the FastAPI application, configures middleware, and includes the top-level router defined in `api/router.py`. The router aggregates versioned endpoints under the `/api/v1` prefix, delegating to individual endpoint modules such as:

- `api/endpoints/projects.py` for creating, listing, and retrieving projects.
- `api/endpoints/pipeline.py` for running the agentic pipeline in batch or streaming mode.
- `api/endpoints/files.py` for browsing the project file tree and fetching or updating file contents.
- `api/endpoints/chat.py` for handling iterative chat interactions tied to a project.
- `api/endpoints/rag.py` and `api/endpoints/sandbox.py` for RAG and sandbox-related operations.

Each endpoint module validates incoming requests using Pydantic schemas from the `schemas` package, calls into dedicated service functions, and returns structured responses. Authentication and authorization concerns are centralized in `auth.py` and integrated into endpoint dependencies where required.

In addition to the core project/pipeline/files/chat endpoints, MetaGPT implements sandbox management endpoints in `api/endpoints/sandbox.py` to support live previews of generated projects. The sandbox workflow is intentionally non-blocking: the `/project_id/create` endpoint returns **202 Accepted** immediately and triggers sandbox creation as a background task, while clients poll `/project_id/status` until the sandbox becomes `ready` and a `preview_url` is available. This design prevents request timeouts during dependency installation and dev-server startup.

4.2.2 Module 2: Agent Orchestration and Graph

The second module is the **Agent Orchestration and Graph Module**, which coordinates the sequence of Manager, Architect, Engineer, and QA agents using LangGraph. The overall pipeline state is defined in `graph/state.py`, where a central data structure captures the prompt, intermediate artifacts, agent outputs, and final project representation.

The orchestration logic in `graph/orchestrator.py` builds a directed graph whose nodes correspond to individual agents and whose edges represent valid transitions. For example, the Manager node takes a raw user prompt and produces a structured requirement specification, which becomes input to the Architect node. The Architect node generates a high-level system design and file plan, which is passed to the Engineer node to create concrete source files. Finally, the QA node inspects the generated artifacts, produces test cases, and records quality-related feedback.

The orchestrator integrates with LangChain's LLM abstractions to call Google Gemini via helper functions defined in the `llm` module. It also records intermediate outputs so that the frontend can display an execution timeline and visualizations of agent reasoning. Error handling, retries, and logging are centralized here, making the pipeline robust and easier to debug or extend with additional agents in the future.

4.2.3 Module 3: Storage and Project Management

The third module is the **Storage and Project Management Module**, which persists all artifacts produced by the pipeline and exposes higher-level operations for managing projects. This module is implemented primarily in the `storage` and `models` packages.

Domain models such as `models/project.py` and `models/user.py` describe core entities, while storage backends in `storage/project_store.py` and `storage/file_store.py` handle reading and writing data. Project state is persisted in **MongoDB** using the Beanie ODM (with Motor as the async driver), as implemented in `db.py`, `models/project.py`, and `storage/project_store.py`. The project store is responsible for:

- Creating a new project directory structure in a configurable `PROJECTS_DIR` for generated code files.
- Recording metadata such as project identifiers, associated prompts, and timestamps in MongoDB.
- Listing existing projects and resolving paths for files within a given project.

The file store provides operations to:

- Generate a tree representation of project files for the frontend file explorer.
- Read file contents for display in the code viewer.
- Apply updates to files when users edit code through the interface.

Together, these components decouple persistent storage concerns from the agent pipeline and API layer. This persistence layer stores project metadata, pipeline status, and agent outputs (including generated files), enabling recovery after server restarts and supporting long-running workflows. When ephemeral disk cache is missing, the pipeline service can restore project files from MongoDB back to disk so that file browsing and preview remain functional.

4.2.4 Module 4: Frontend User Interface

The fourth module is the **Frontend User Interface**, implemented with Next.js, TypeScript, and Tailwind CSS under the `frontend/src` directory. It provides an interactive, dark-themed workspace where users can manage projects and visualize the behavior of the agent pipeline.

The App Router structure in `src/app` defines key pages:

- `page.tsx` as the landing page for entering prompts and viewing recent projects.
- `projects/page.tsx` for listing all projects retrieved from the backend.
- `project/[id]/page.tsx` as the main workspace for a specific project, showing files, agent outputs, and chat.
- `signin/page.tsx` and `signup/page.tsx` for authentication flows (if enabled).

Reusable components in `src/components` encapsulate major UI features:

- `FileExplorer.tsx` renders the project tree and allows users to select files.
- `CodeViewer.tsx` displays syntax-highlighted source code and possibly supports editing.
- `AgentOutputs.tsx` shows structured reasoning and outputs from each agent in the pipeline.
- `ExecutionTimeline.tsx` visualizes the sequence and status of agent executions.
- `ChatPanel.tsx` enables iterative conversations tied to a particular project.

Client-side state is managed using stores defined in `src/lib/store.ts` and related files, while API communication is abstracted in `src/lib/api.ts`. Global styles and Tailwind configuration ensure a consistent look-and-feel across the application.

4.2.5 Module 5: LLM, SOP, and Retrieval

The fifth module is the **LLM, SOP, and Retrieval Module**, which encapsulates all logic related to configuring the underlying LLM, defining agent SOPs, and supporting retrieval-augmented generation (RAG). This module ties together the conceptual behavior of agents with concrete model calls and knowledge sources.

The `llm/gemini.py` file provides helper functions such as `get_llm` and `get_llm_with_structured_output`, which create LangChain-compatible LLM instances backed by Google Gemini. Configuration values such as model name, temperature, maximum tokens, and API key are read from `config.py` and environment variables, ensuring that model behavior can be tuned without changing application code.

Agent behaviors are specified in `sop/definitions.py`, where each agent's role, objectives, inputs, outputs, constraints, and quality checklist are documented. These SOPs are consumed by the agent implementations in the `agents` package, providing a clear contract between high-level design and concrete prompts issued to the LLM.

The RAG submodule in `rag` contains utilities for indexing and retrieving relevant documents. Files such as `rag/indexer.py`, `rag/vector_store.py`, `rag/retriever.py`, and `rag/embeddings.py` manage the creation of vector embeddings, storage of vectors, and retrieval of context for augmenting LLM prompts. This enables MetaGPT to ground agent reasoning in existing project artifacts or external knowledge bases, improving accuracy and consistency for complex tasks.

4.3 Data Flow Diagram

Data Flow Diagrams (DFDs) illustrate the movement of data within the MetaGPT system and how information flows between different agents and components.

4.3.1 Context Diagram (Level 0 DFD)

The Level 0 DFD illustrates the high-level interaction between the User and the MetaGPT system, showing the complete workflow from user input to final code delivery.

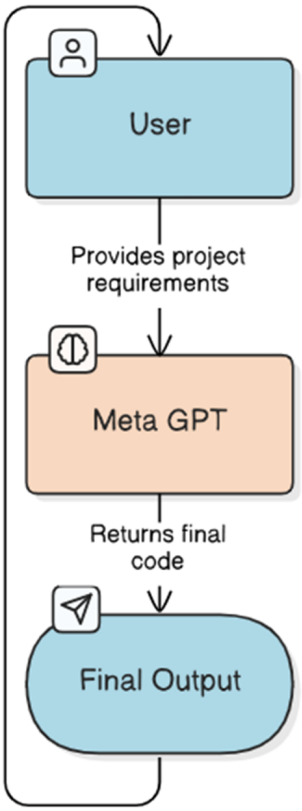

Figure 4.3: DFD Level 0 - MetaGPT System

4.3.2 Level 1 DFD

The Level 1 DFD illustrates the internal workings of the MetaGPT system, showing how specialized agents collaborate to transform user requirements into validated code output through structured workflows and iterative feedback loops.

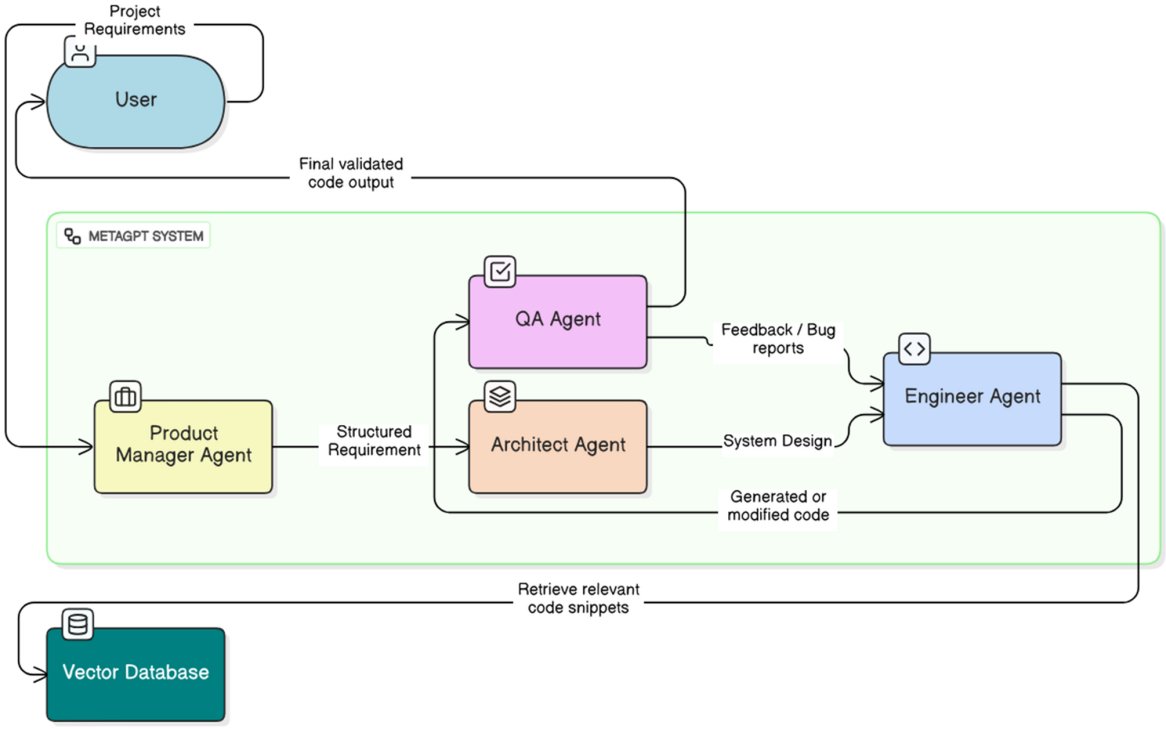

Figure 4.4: DFD Level 1 - MetaGPT System Internal Workflow

4.3.3 Level 2 DFD

The Level 2 Data Flow Diagram provides a detailed decomposition of the MetaGPT system's internal agent collaboration process, showing the granular data flows and transformations that occur within each agent's specialized workflow.

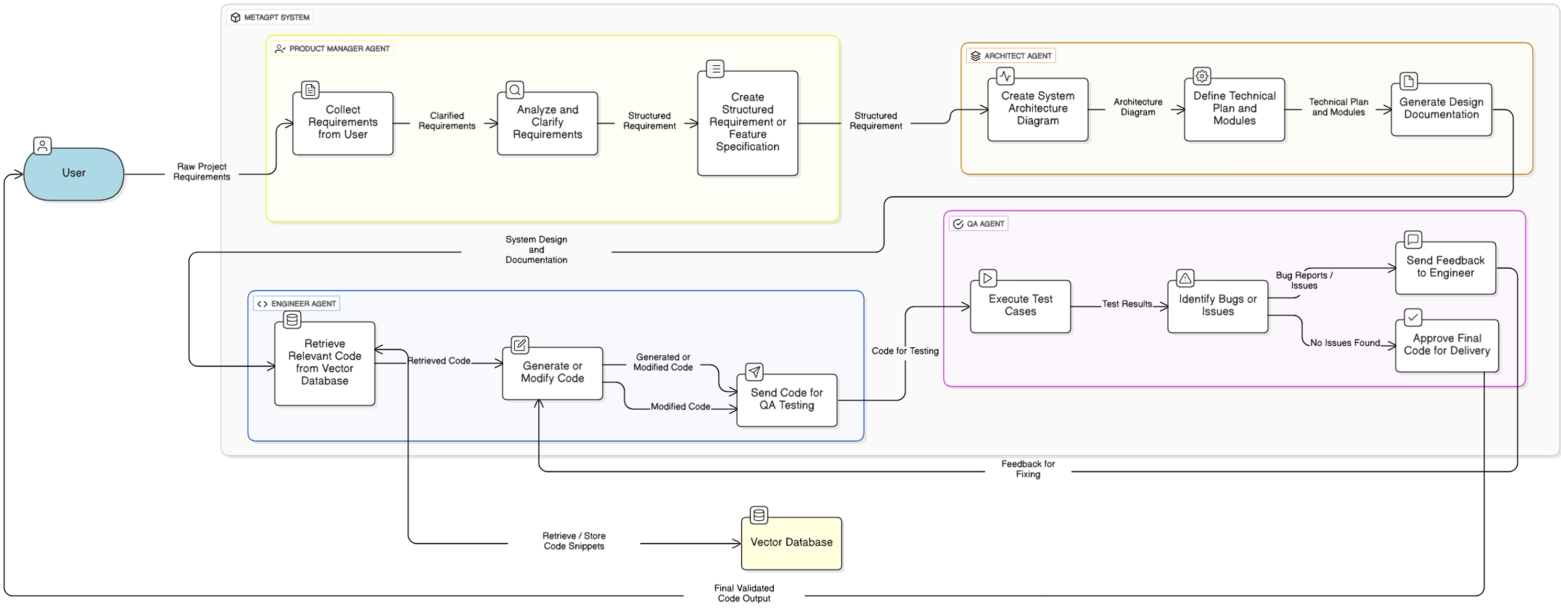

Figure 4.5: DFD Level 2 — Detailed Agent Collaboration Process

4.4 ER Diagram

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

4.5 UI Design

The User Interface (UI) is designed to be clean, modern, and intuitive, prioritizing the agent management and workflow visualization experience. The following screenshots illustrate the key screens of the application.

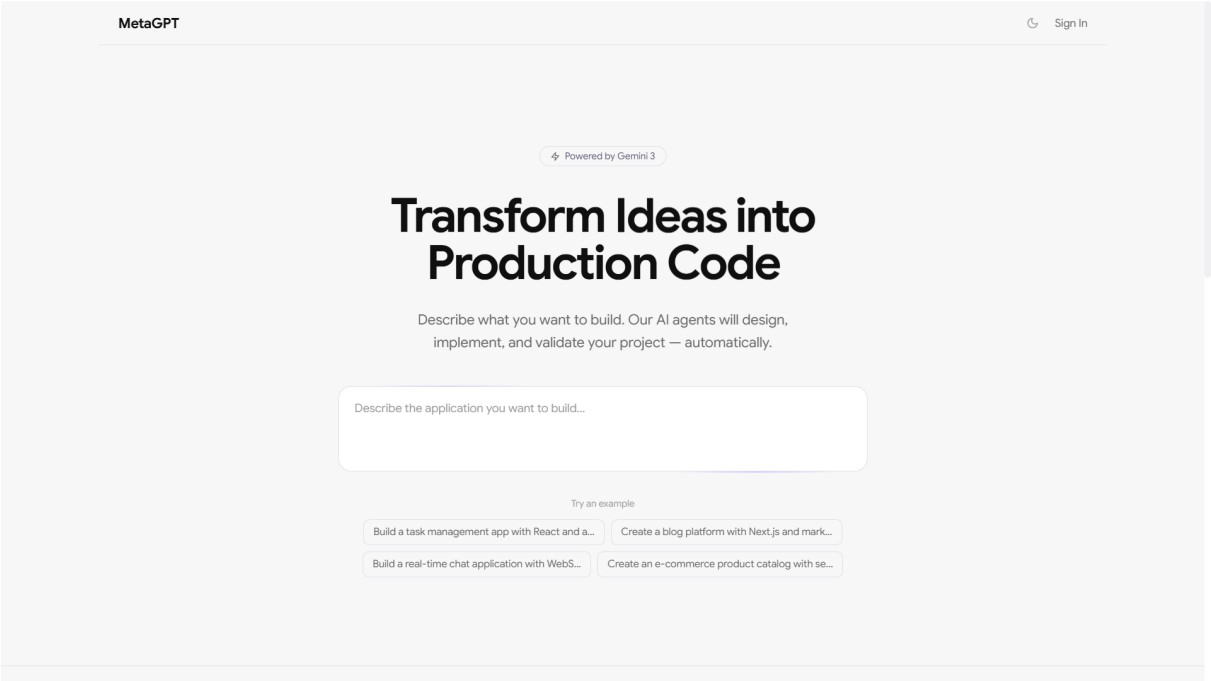

Figure 4.6: MetaGPT — Home Page

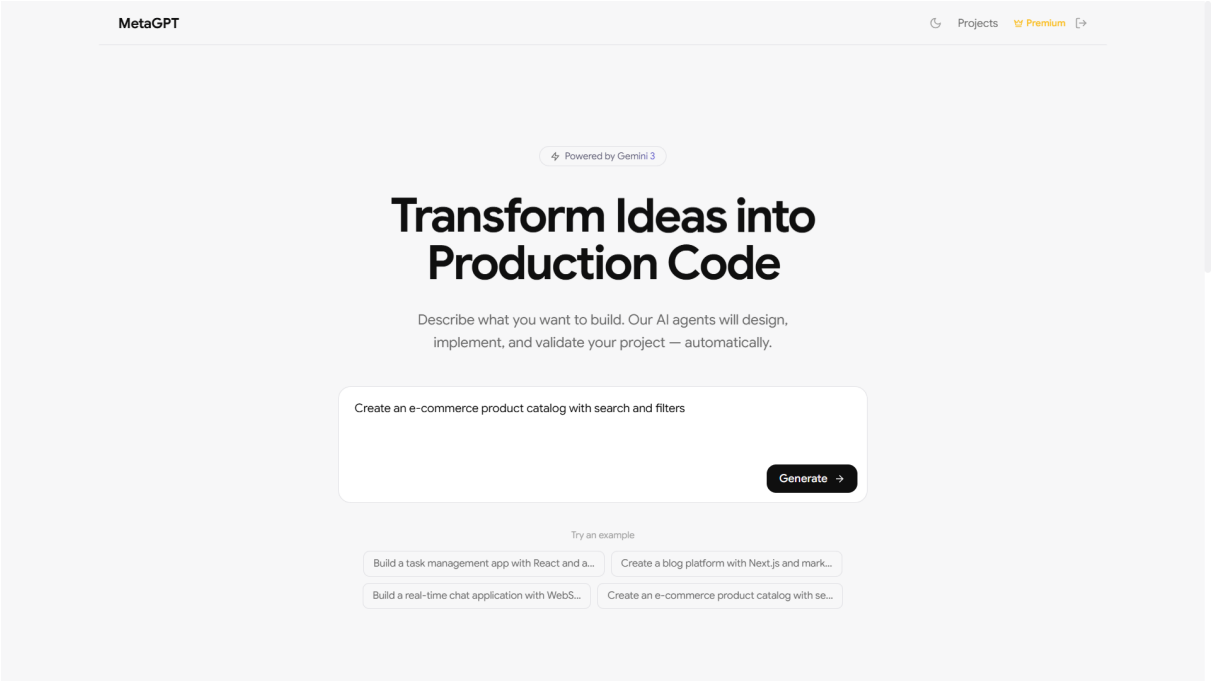

Figure 4.7: MetaGPT — Home Page with Prompt Entered

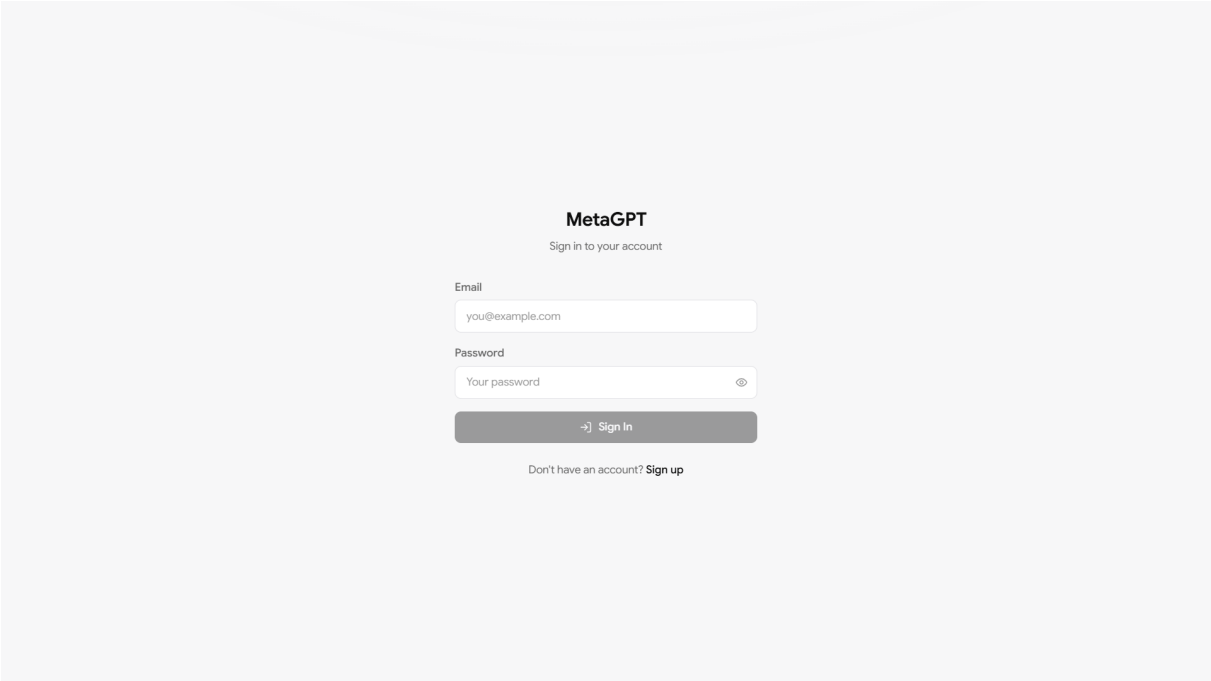

Figure 4.8: MetaGPT — Sign In Page

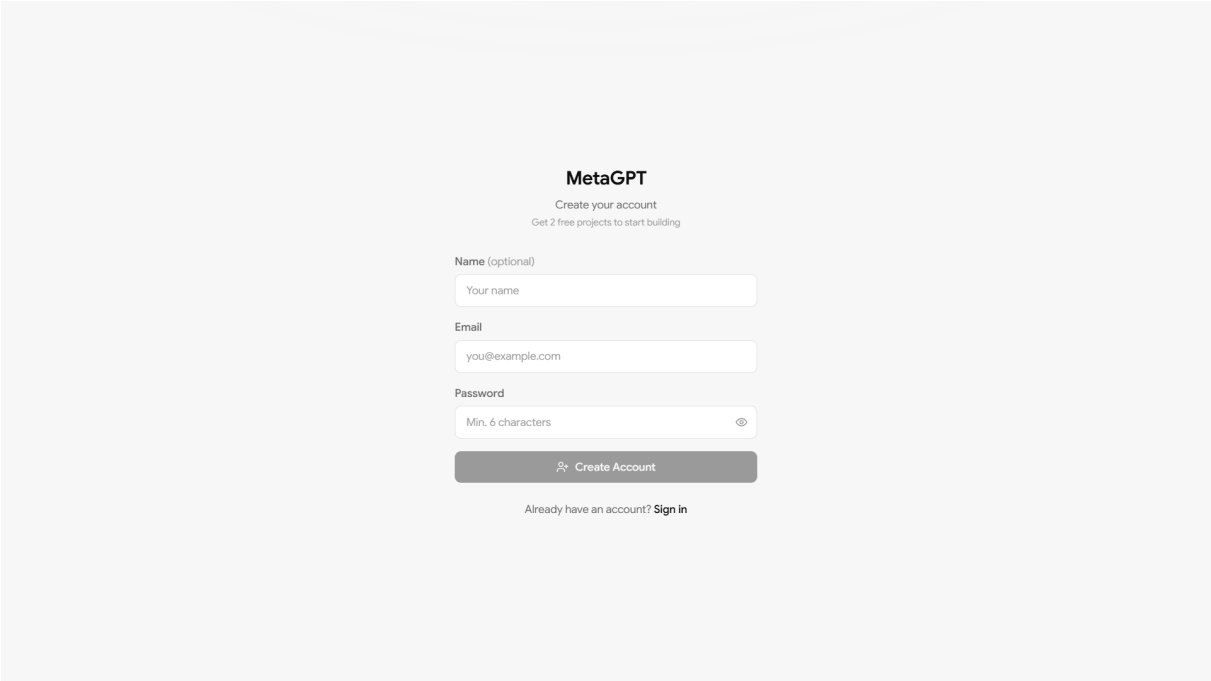

Figure 4.9: MetaGPT — Sign Up Page

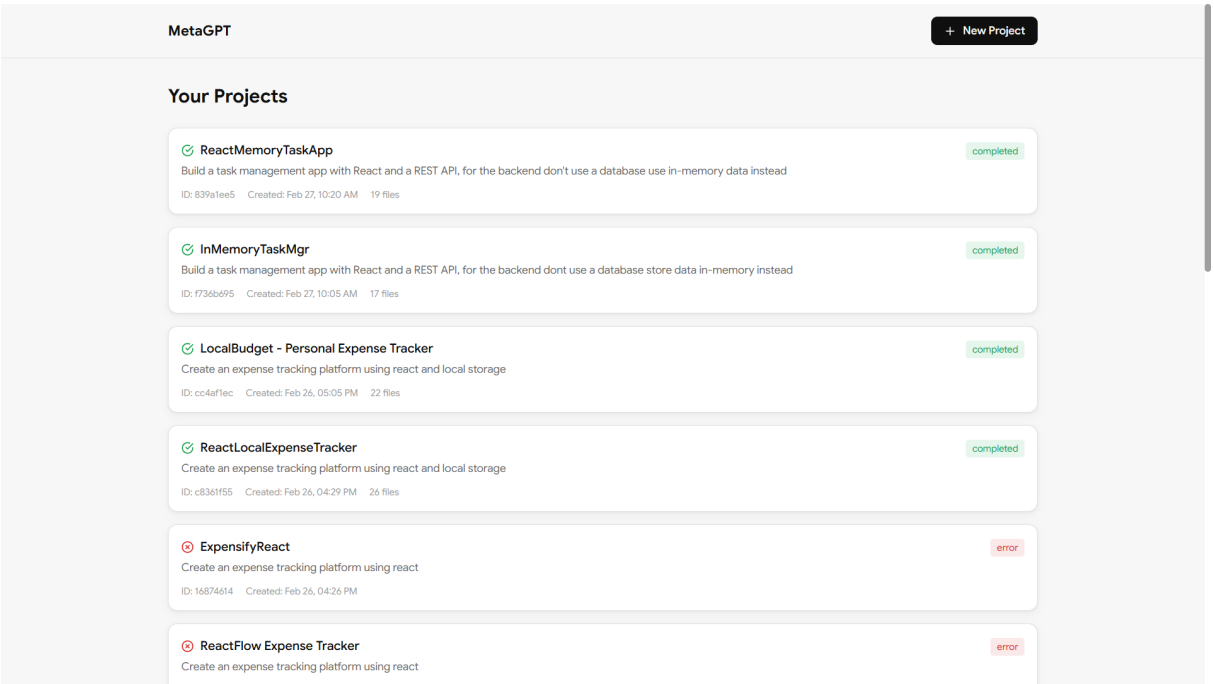

Figure 4.10: MetaGPT — List of Projects

Figure 4.11: MetaGPT — Manager Agent Output

Manager

Project Name	**LocalBudget - Personal Expense Tracker**
Type	**web-app**
Tech Stack	**React (Vite recommended), TypeScript, Tailwind CSS, React Context API, LocalStorage API, Chart.js (optional for visualization), uuid (for unique ID generation)**

Description

A client-side Single Page Application (SPA) designed for personal finance management. The application allows users to track their daily expenses, categorize spending, and visualize total costs. It operates entirely within the browser using React for the UI and LocalStorage for data persistence, ensuring privacy and offline capability without the need for a backend server.

Constraints

- Must use React as the frontend framework.
- Must use LocalStorage for database/persistence (No backend API).
- Data is limited to the storage quota of the browser (typically ~5MB for LocalStorage).
- Application is single-user only (tied to the specific browser/device).

Assumptions

- The user does not require multi-device synchronization.
- Currency will be hardcoded to a default (e.g., USD) or set once in a simple setting.
- Authentication is not required as access is controlled by physical access to the device.
- Historical data analysis (charts/graphs) is a 'nice to have' but not a core requirement for MVP.

Requirements (6)

FR-01 high functional
Expense Entry Form: Users must be able to input expense details including title, amount, date, and category.
> Show acceptance criteria (3)

FR-02 high functional
Expense Listing: Users can view a list of all recorded expenses.
> Show acceptance criteria (3)

FR-03 medium functional
Dashboard Summary: Users see a summary of their total spending.
> Show acceptance criteria (2)

FR-04 medium functional
Edit and Delete Operations: Users can modify or remove existing entries.
> Show acceptance criteria (3)

NFR-01 high non-functional
Data Persistence: Application state must survive browser refreshes.
> Show acceptance criteria (3)

> +1 more requirements

> Show reasoning

Figure 4.12: MetaGPT — Architect Agent Output

Architect

Architecture	**Client-Side Component-Based Architecture (SPA)**
Components	**8 components**
Files	**21 files**

File Structure

`index.html`
Root HTML file

`package.json`
Project metadata and dependencies

`tsconfig.json`
TypeScript config

`vite.config.ts`
Vite bundler config

`tailwind.config.js`
Tailwind styling config

> +16 more files

API Design

Context Interface

```
state: expenses: Expense[]
addExpense: (expense: Omit<Expense, 'id'>) => void
deleteExpense: (id: string) => void
editExpense: (id: string, updatedExpense: Omit<Expense, 'id'>) => void
getExpense: (id: string) => Expense | undefined
```

> Database Schema

Deployment Notes

Since this is a client-side only application, it can be deployed to any static host (Netlify, Vercel, GitHub Pages). Development: Run `npm run dev`. Ensure Vite is configured to open or expose port 5173 (default). Build: Run `npm run build` to generate the `dist` folder. Note on Persistence: Data is stored in the user's browser LocalStorage. Clearing browser cache will delete the data.

Components

> **App Root** frontend

> **State Management (Context)** service

> **Custom Hooks** utility

> **Layout Components** frontend

> **Dashboard Components** frontend

> +3 more components

Data Flow

1. App Initialization: `App.tsx` renders `ExpenseProvider`. `ExpenseProvider` initializes state using `useLocalStorage`. If LocalStorage has data, it populates the state; otherwise, it starts with an empty array. 2. Reading Data: `ExpenseList` and `DashboardSummary` consume `ExpenseContext`. `ExpenseList` maps over the `expenses` array to render `ExpenseItem` components. `DashboardSummary` reduces the array to calculate the total. 3. Creating Data: User submits `ExpenseForm`. The form calls `addExpense` from Context. The Context updates the state array and generates a UUID. `useLocalStorage` detects the state change and writes the new JSON array to the browser's LocalStorage. 4. Updating/Deleting: User clicks Edit/Delete in `ExpenseItem`. Delete calls `deleteExpense` in Context. Edit populates `ExpenseForm` (state lifted or passed via prop/context) which then calls `editExpense` on submission. State updates trigger LocalStorage sync.

> Show reasoning

Figure 4.13: MetaGPT — Engineer Agent Output

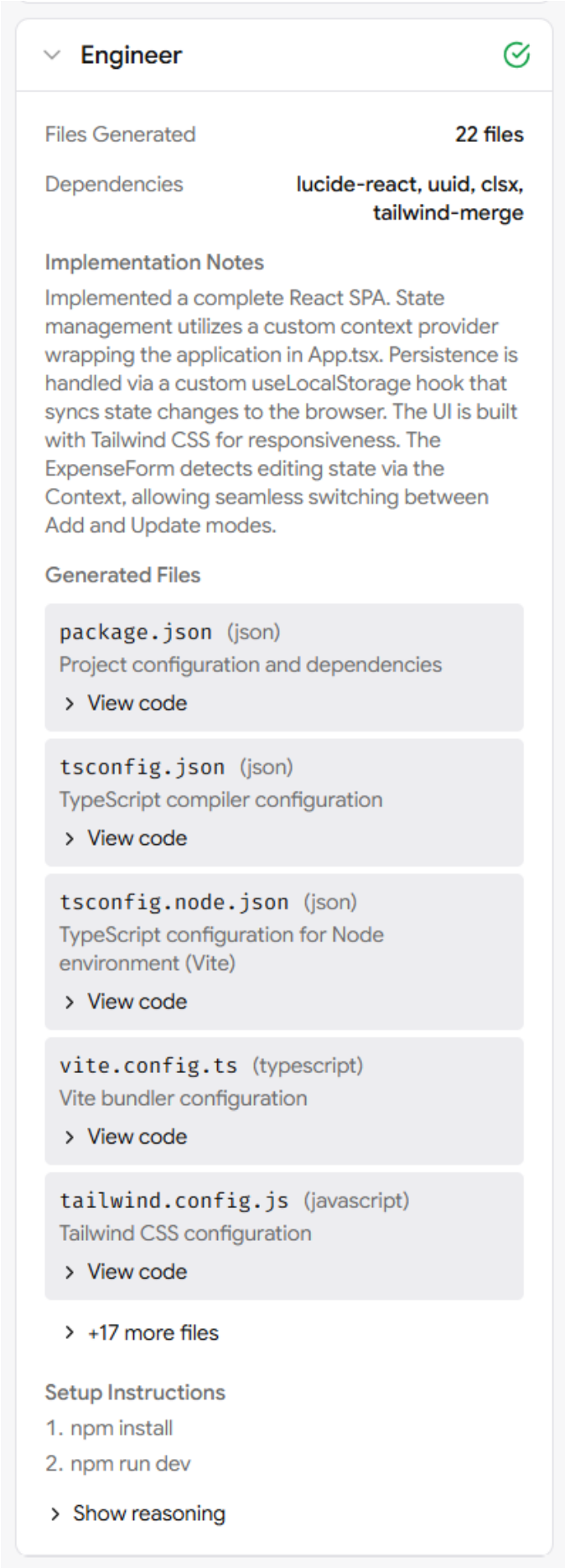

Figure 4.14: MetaGPT — QA Agent Output

QA

Quality Score 80/100 ⚠ needs-revision

Test Coverage 85%

Test Cases 6 tests

Test Cases

TC-UNIT-01 Expense Context - Add Expense

(unit)

Verify that the addExpense function correctly updates the state.

`target: src/context/ExpenseContext.tsx`

Expected: The expenses array should contain the new expense object with a generated UUID.

> View test code

TC-MANUAL-01

Form Validation - Negative Amount (manual)

Verify that the form prevents submission of negative numbers as per FR-01.

`target: src/components/Expenses/ExpenseForm.tsx`

1. Open the application.
2. Enter 'Test Item' in Title.
3. Enter '-50' in Amount.
4. Select a valid date and category.
5. Click 'Add Expense'.

Expected: The form should NOT submit, and an error message should indicate that amounts must be positive.

TC-E2E-01

Complete Expense Lifecycle (CRUD) (e2e)

Verify adding, viewing, editing, and deleting an expense.

`target: src/App.tsx`

1. Fill out the Expense Form with valid data and submit.
2. Verify the new item appears in the Expense List at the top.
3. Verify the Dashboard Total updates correctly.
4. Click the Edit icon on the item.
5. Change the amount and Save.
6. Verify the item updates in the list and Dashboard.
7. Click the Delete icon and confirm.
8. Verify the item is removed from the list.

Expected: All operations complete successfully without page reload; data persists correctly.

TC-INT-01 Data Persistence on Reload

(integration)

Verify data remains after browser refresh (NFR-01).

`target: src/hooks/useLocalStorage.ts`

1. Add a unique expense item 'Persistence Test'.
2. Refresh the browser page.
3. Check the Expense List.

Expected: The 'Persistence Test' item should still be visible in the list.

TC-MANUAL-02 Date Display Accuracy

(manual)

Verify date shown matches selected date regardless of timezone.

`target: src/components/Expenses/ExpenseItem.tsx`

Figure 4.15: MetaGPT — QA Agent Output (continued)

1. Select a specific date (e.g., 1st of month) in the form.
2. Submit the expense.
3. Observe the date day displayed in the list item icon.

Expected: The displayed day number matches the input date exactly, ensuring no off-by-one errors due to timezone offsets.

> +1 more tests

Validation Notes

src/components/Expenses/ExpenseForm.tsx · functionality

Missing negative amount validation. The requirement [FR-01] states validation must prevent negative amounts. The current check `if (!title.trim() || !amount || !date)` only checks for truthiness/existence.

Update handleSubmit validation logic to include `if (parseFloat(amount) <= 0) return;` or add HTML attribute `min='0'` to the input.

src/components/Expenses/ExpenseItem.tsx · maintainability

Fragile Date Calculation. The code `new Date(expense.date).getDate() + 1` is a hardcoded fix for timezone offsets that may fail for users in positive timezones (East of UTC), potentially displaying the wrong date.

Use the existing `formatDate` utility which handles timezone offsets correctly, or use `getUTCDate()` if the input string is YYYY-MM-DD, rather than manually adding +1 to the local date.

src/components/Expenses/ExpenseList.tsx · functionality

Date Sorting Logic. The sort function `new Date(a.date).getTime() - new Date(b.date).getTime()` instantiates Date objects repeatedly inside the sort loop, which is inefficient, though acceptable for small lists.

For better performance on large lists, compare the ISO string directly (e.g. `a.date.localeCompare(b.date)`) since ISO dates are lexically sortable.

src/hooks/useLocalStorage.ts · performance

Synchronous LocalStorage calls. LocalStorage operations are synchronous and run on the main thread. If the expense list grows very large, this could cause UI stutter during saves.

For a production app, consider debouncing the save operation or using IndexedDB for larger datasets.

Code Review Summary

The application successfully implements the core requirements for a local expense tracker using React and LocalStorage. The architecture is clean, utilizing Context API for state management and custom hooks for persistence. The UI is well-structured with Tailwind CSS. However, there are critical functionality gaps regarding input validation (negative numbers) and potentially buggy date handling logic in the list view that relies on 'magic numbers' (+1) to fix timezone issues. These need to be addressed to fully meet the Acceptance Criteria.

> Show reasoning

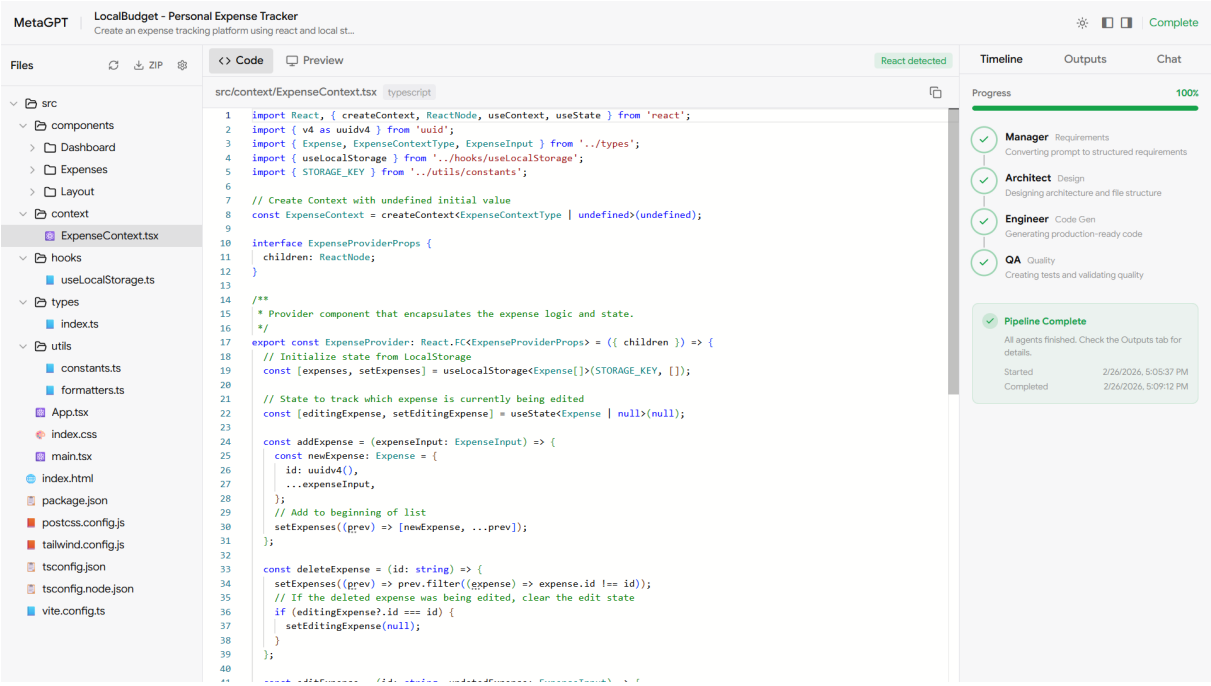

Figure 4.16: MetaGPT — Code Preview

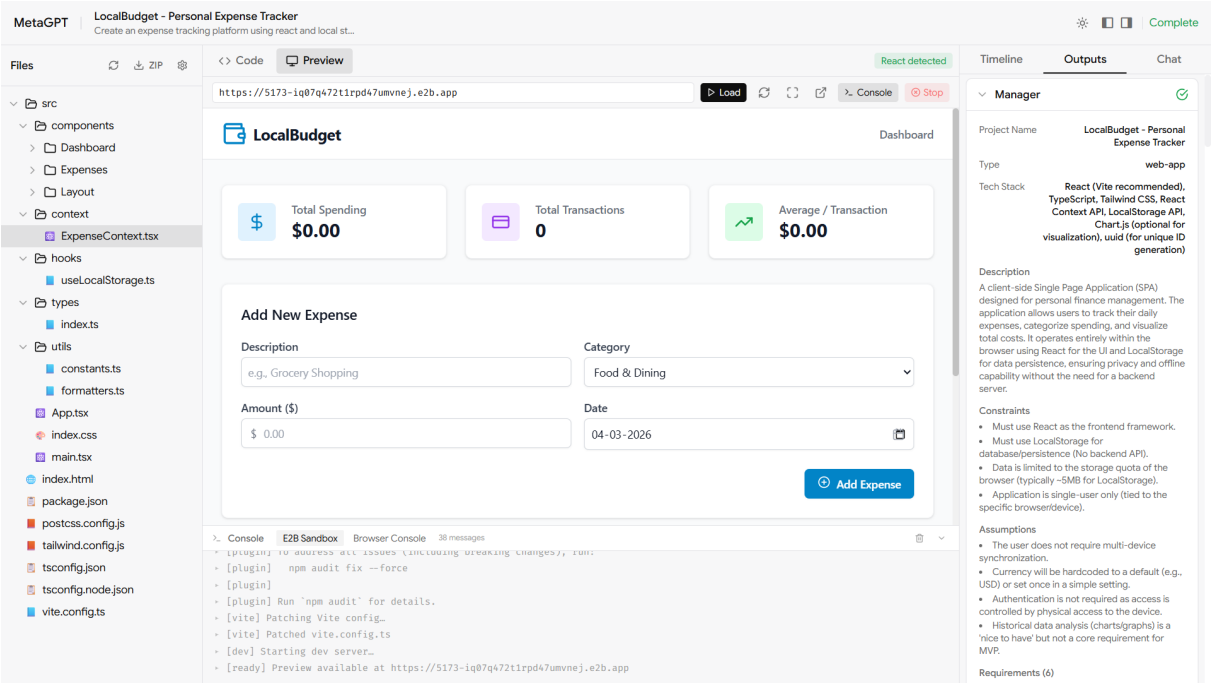

Figure 4.17: MetaGPT — Sandbox Live Preview

Chapter 5

Implementation And Results

5.1 Software And Hardware Requirements

The implementation of MetaGPT relies on a modern full-stack environment with the following software and hardware requirements:

- **Hardware**: A development machine with a multi-core processor, at least 4 GB of RAM, and sufficient disk space to store generated projects. For typical experiments, a standard laptop or desktop configuration is sufficient.
- **Operating System**: Any operating system capable of running Python 3.11 and Node.js 18 or above (e.g., Windows, Linux, or macOS).
- **Backend Software**: Python 3.11+, FastAPI, LangChain, LangGraph, and supporting libraries managed using the `uv` package manager or `pip`. A valid Google API key is required to access the Gemini model.
- **Frontend Software**: Node.js 18+, Next.js 14, TypeScript, Tailwind CSS, and associated tooling such as ESLint and TypeScript for linting and type checking.
- **Tools**: Git for version control, a terminal or shell for running backend and frontend servers, and an editor or IDE for browsing code and logs.

5.2 Implementation Details

Module 1: Backend API Layer Implementation

The backend is organized under `backend/app` and exposes its functionality through a FastAPI application defined in `main.py`. In this file, the FastAPI instance is created, CORS and other middleware are configured, and the top-level router from `api/router.py` is included, as shown in Listing 5.1. The router groups all versioned endpoints under the `/api/v1` prefix and delegates to modules in `api/endpoints`.

```python
def create_app() -> FastAPI:
    settings = get_settings()
    app = FastAPI(
        title=settings.app_name,
        version=settings.app_version,
        lifespan=lifespan,
    )
    # Configure CORS
    app.add_middleware(
        CORSMiddleware,
        allow_origins=settings.cors_origins,
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )
    # Include API routes
    app.include_router(api_router, prefix=settings.api_prefix)
    return app
```

Listing 5.1: FastAPI application configuration and middleware setup

Listing 5.2 shows a representative endpoint for creating a new project. It validates user credits, calls the pipeline service, and handles persistence.

```python
@router.post("", response_model=Project, status_code=status.HTTP_201_CREATED)
async def create_project(
    request: ProjectCreate,
    user: User = Depends(get_current_user),
) -> Project:
    if not user.has_credits():
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="You've used all your free credits. Upgrade to premium.",
        )
    service = PipelineService()
    project = await service.create_project(request.prompt, user_id=str(user.id))
    user.credits_used += 1
    await user.save()
    return project
```

Listing 5.2: FastAPI endpoint for project creation

The implementation of individual endpoints is split into focused modules:

- `api/endpoints/projects.py` implements handlers for creating new projects, listing existing projects, and retrieving project metadata. It uses request/response models from `schemas/projects.py`.
- `api/endpoints/pipeline.py` contains endpoints to start a pipeline run (both standard and streaming) and to query pipeline status or artifacts, delegating to functions in `services/pipeline_service.py`.
- `api/endpoints/files.py` provides a tree view of project files and operations to read or update file contents via the storage layer (`storage/file_store.py`).

- `api/endpoints/chat.py` implements chat-based iteration endpoints that forward messages to `services/chat_service.py`.
- `api/endpoints/rag.py` and `api/endpoints/sandbox.py` expose retrieval and sandbox-related operations.

Cross-cutting concerns such as authentication and authorization are implemented in `auth.py` and reused across endpoints using FastAPI dependencies. Configuration values required by the API layer are loaded from `config.py`, which reads from environment variables and provides typed settings via Pydantic.

E2B Sandbox preview endpoints: The sandbox endpoint module (`api/endpoints/sandbox.py`) is designed for long-running preview builds. It implements a non-blocking create flow that returns **202 Accepted** immediately and starts a background task that writes generated files, installs dependencies, and starts dev servers inside an isolated E2B sandbox. Users (via the frontend) poll the status endpoint until the sandbox becomes `ready`, at which point a public `preview_url` is returned for embedding in an iframe.

Module 2: Agent Orchestration and Graph Implementation

Agent orchestration is implemented in the `graph` and `agents` packages. The central pipeline state is defined in `graph/state.py`, where a Pydantic model captures the user prompt, derived requirements, architectural plan, generated files, and QA feedback. This state object is passed between nodes in a LangGraph graph.

The graph itself is constructed in `graph/orchestrator.py`. Here, nodes corresponding to the Manager, Architect, Engineer, and QA agents are registered, and edges are defined to specify the order of execution, as demonstrated in Listing 5.3. Each node invokes a concrete agent class from the `agents` package:

```python
def _build_graph(self) -> StateGraph:
    workflow = StateGraph(PipelineState)
    # Add nodes for each agent
    workflow.add_node("manager", self._run_manager)
    workflow.add_node("architect", self._run_architect)
    workflow.add_node("engineer", self._run_engineer)
    workflow.add_node("qa", self._run_qa)

    # Define deterministic linear flow
    workflow.set_entry_point("manager")
    workflow.add_edge("manager", "architect")
    workflow.add_edge("architect", "engineer")
    workflow.add_edge("engineer", "qa")
    workflow.add_edge("qa", END)

    return workflow.compile()
```

Listing 5.3: LangGraph orchestrator pipeline configuration

- `agents/manager.py` receives the raw prompt and produces structured requirements.
- `agents/architect.py` designs the system architecture and file layout based on those requirements.

- `agents/engineer.py` generates implementation code for the planned files. Listing 5.4 illustrates how the Engineer Agent builds its LLM prompt using outputs from previous stages and SOP constraints.
- `agents/qa.py` inspects artifacts and produces test cases and quality feedback.

```python
def _build_prompt(self, **inputs: Any) -> str:
    architect_output: ArchitectOutput = inputs.get("architect_output")
    manager_output: ManagerOutput = inputs.get("manager_output")

    # Format SOP sections
    constraints_text = self._format_sop_section(self.sop.constraints)
    quality_text = self._format_sop_section(self.sop.quality_checklist)

    # Format architecture details
    file_structure = self._format_file_structure(architect_output)
    components = self._format_components(architect_output)
    requirements = self._format_requirements(manager_output)

    return f"""# Engineer Agent - Code Generation
## Your Role
{self.sop.role}

## Constraints
{constraints_text}

## Project: {manager_output.project_name}
{manager_output.project_description}

## File Structure:
{file_structure}

## Your Task
Implement ALL files listed in the file structure with production-ready code.
"""
```

Listing 5.4: Engineer Agent building its prompt with context from earlier pipeline stages

All agents inherit from a base class in `agents/base.py`, which standardizes how LLM calls are made, how inputs/outputs are validated using `schemas/agents.py`, and how logging is performed. The orchestrator integrates with LangChain's LLM abstractions provided by `llm/gemini.py`, enabling both free-form and structured-output interactions. The final orchestration API is exposed to the rest of the application through `services/pipeline_service.py`, which provides functions to start runs, stream intermediate events, and fetch stored artifacts.

Module 3: Storage and Project Management Implementation

Storage concerns are encapsulated in the `models` and `storage` packages. Domain models in `models/project.py` and `models/user.py` describe how projects and users are represented in memory, including fields such as identifiers, names, prompts, and timestamps. These models are mirrored by Pydantic schemas in `schemas/projects.py` to ensure consistency between API payloads and internal representations.

Project-level persistence is handled by `storage/project_store.py`. This module is responsible for creating project directories under the configured `PROJECTS_DIR`, writing metadata files

if needed, and enumerating existing projects. File-level operations are implemented in `storage/file_store.py`, which provides functions to:

- Walk a project directory and build a hierarchical tree structure consumed by the frontend `FileExplorer`.
- Read file contents from disk and return them as text for display or further processing.
- Apply write operations when agents or users modify files, ensuring that paths are validated and confined to the project root.

These storage modules are consumed by both the API endpoints (for serving file trees and contents) and the agent pipeline (for writing generated code to disk). By centralizing filesystem access in a small number of modules, the system can later be extended to support alternative backends such as databases or cloud storage with minimal changes elsewhere.

MongoDB persistence: In our framework, project state is stored in MongoDB. The database connection is initialized in `db.py` using Motor and Beanie, and document models such as `models/project.py` and `models/user.py` are registered with Beanie. The `ProjectStore` (`storage/project_store.py`) provides CRUD operations to create projects, update pipeline status, and persist agent outputs (Manager/Architect/Engineer/QA). Listing 5.5 shows how a new project document is initialized and persisted. This database-backed state allows the application to restore generated files to disk when ephemeral storage is lost (e.g., after container restart), ensuring that the file explorer and subsequent operations remain consistent.

```
async def create(self, project_id: str, prompt: str, user_id: str = "") -> Project:
    doc = ProjectDocument(
        project_id=project_id,
        user_id=user_id,
        prompt=prompt,
        state=ProjectState(
            pipeline_status=PipelineStatus(
                stage=PipelineStage.PENDING,
                progress=0,
            )
        ),
        created_at=datetime.utcnow(),
        updated_at=datetime.utcnow(),
    )
    await doc.insert()
    return self._to_project(doc)
```

Listing 5.5: Creating and persisting a project document using Beanie ODM

Module 4: Frontend User Interface Implementation

The frontend implementation lives in the `frontend/src` directory and is based on the Next.js App Router. Routing and layout are defined in `src/app/layout.tsx` and the various page components:

- `src/app/page.tsx` implements the landing page where users can provide prompts and see high-level information about the system. A snippet handling the submission of a new project prompt is shown in Listing 5.6.

- `src/app/projects/page.tsx` fetches the list of projects from the backend and renders them in a tabular or card-based layout.
- `src/app/project/[id]/page.tsx` is the main workspace, composed of panels for the file explorer, code viewer, agent outputs, execution timeline, and chat.

```
const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!prompt.trim() || isLoading) return;
    if (!token) {
        router.push("/signin");
        return;
    }
    setIsLoading(true);
    try {
        const { createProject } = await import("@/lib/api");
        const project = await createProject(prompt.trim());
        router.push(`/project/${project.id}`);
    } catch (error: unknown) {
        setSubmitError("Failed to create project");
        setIsLoading(false);
    }
};
```

Listing 5.6: Client-side form submission handler in Next.js

Key interactive components are implemented in `src/components`:

- `FileExplorer.tsx` renders the tree returned by the files API and triggers file selection events. When only a flat list of generated files is available (e.g., streaming in real-time), it dynamically reconstructs the hierarchical folder structure as demonstrated in Listing 5.7.
- `CodeViewer.tsx` uses a code editor or viewer (e.g., Monaco) to display and possibly edit source files.
- `AgentOutputs.tsx` subscribes to pipeline events and presents agent reasoning and outputs in a structured way.
- `ExecutionTimeline.tsx` visualizes the order and status of agent executions for a given run.
- `ChatPanel.tsx` provides a text area and message list for iterative refinement of a project as shown handling submissions in Listing 5.8.

```
function buildTreeFromFiles(files: { file_path: string }[]): FileTreeNode | null {
  if (files.length === 0) return null;

  const root: FileTreeNode = { name: "files", path: "/", type: "directory", children:
       [], };

  for (const file of files) {
    const parts = file.file_path.split("/").filter(Boolean);
    let current = root;

    for (let i = 0; i < parts.length; i++) {
      const part = parts[i];
      const isFile = i === parts.length - 1;
      const path = parts.slice(0, i + 1).join("/");
      let child = current.children?.find((c) => c.name === part);

      if (!child) {
        child = { name: part, path, type: isFile ? "file" : "directory", children:
            isFile ? [] : [], };
        current.children = current.children || [];
        current.children.push(child);
      }
      current = child;
    }
  }
  return root;
}
```

Listing 5.7: Dynamically building a file tree structure from a flat list of file paths

The module `src/lib/api.ts` centralizes HTTP calls to the backend, while `src/lib/store.ts` and related files implement client-side state management (e.g., current project, selected file, pipeline status). Tailwind CSS classes and global styles in `src/app/globals.css` are used to implement the dark theme and responsive layout.

```
const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    if (!message.trim() || isLoading) return;

    const userMessage = message.trim();
    setMessage("");
    setIsLoading(true);

    try {
        const response = await sendChatMessage(
            projectId,
            userMessage,
            autoMode ? null : selectedModel,
        );

        addChatMessage({
            ...response.message,
            files_referenced: response.files_referenced,
            indexing_status: response.indexing_status,
        });

        // If indexing is pending, trigger it live
        if (response.indexing_status === "pending") {
            indexProjectFiles(projectId, response.files_modified);
        }
    } finally {
        setIsLoading(false);
    }
};
```

Listing 5.8: Chat handler supporting Server-Sent Events (SSE) and background indexing

Module 5: LLM, SOP, and Retrieval Implementation

The LLM, SOP, and retrieval logic is implemented in the `llm`, `sop`, and `rag` packages. The file `llm/gemini.py` exposes functions such as `get_llm()` and `get_llm_with_structured_output(schema)` (see Listing 5.9). These functions read configuration from `config.py` (e.g., model name, temperature, API key) and return LangChain `ChatModel` instances configured for MetaGPT. This ensures that all agents share a single, centrally managed LLM configuration.

```
def get_llm_with_structured_output(
    output_schema: type[T],
    temperature: float | None = None,
    max_tokens: int | None = None,
    model: str | None = None,
) -> BaseChatModel:
    base_llm = get_llm(temperature=temperature, max_tokens=max_tokens, model=model)
    return base_llm.with_structured_output(output_schema)
```

Listing 5.9: Configuring Gemini for structured JSON output via LangChain

Agent SOPs are defined in `sop/definitions.py`. Each SOP includes fields such as role description, objectives, required inputs, expected outputs, constraints, and a quality checklist. The concrete agent classes in `agents/*.py` load these SOPs and use them to construct prompts

or system messages when calling the LLM, ensuring that behavior is consistent and traceable across runs.

Retrieval-augmented generation is supported by the `rag` package:

- `rag/embeddings.py` configures embedding models used to convert text into vector representations.
- `rag/vector_store.py` manages storage and querying of embeddings in a vector database or in-memory index.
- `rag/indexer.py` defines workflows for indexing new documents, such as project files or external resources.
- `rag/retriever.py` implements retrieval logic that selects the most relevant chunks of text to be added as context to LLM prompts. Listing 5.10 highlights the core similarity search function.

```python
async def retrieve(self, project_id: str, query: str, k: int | None = None) -> list[RetrievedChunk]:
    if not self.settings.rag_enabled:
        return []

    k = k or self.settings.rag_top_k
    try:
        vector_store = get_vector_store(project_id)
        # Similarity search with scores
        results = vector_store.similarity_search_with_relevance_scores(query, k=k)

        chunks = []
        for doc, score in results:
            chunks.append(
                RetrievedChunk(
                    content=doc.page_content,
                    file_path=doc.metadata.get("file_path", "unknown"),
                    language=doc.metadata.get("language", "text"),
                    relevance_score=score,
                )
            )
        return chunks
    except Exception as e:
        logger.error(f"Retrieval error for project {project_id}: {e}")
        return []
```

Listing 5.10: Retrieving relevant codebase chunks from the vector store

By combining these pieces, the system can ground agent reasoning in both the current project state and external knowledge sources, improving the relevance and reliability of generated code and explanations.

5.3 Testing

5.3.1 Unit Testing

Table 5.1 summarizes representative unit test cases for the main modules of MetaGPT. Each test case specifies the module under test, the scenario being validated, sample input, expected result, and the final status of execution.

Table 5.1: Unit Test Cases for MetaGPT Modules

Module	**Test Scenario**	**Input**	**Expected Result**	**Status**
Backend API Layer	Load configuration with valid environment variables	Populated `.env` file (e.g., `GOOGLE_API_KEY`, `PROJECTS_DIR`)	`Config` object created successfully with all fields set and no validation errors	Success
Backend API Layer	Handle missing optional configuration values	`.env` file without optional fields (e.g., no `LLM_TEMPERATURE`)	Default values applied for missing options; application starts without raising exceptions	Success
Backend API Layer	Validate project creation endpoint	HTTP `POST /api/v1/projects` with a valid JSON body containing a prompt	Response status `201`; response body contains non-empty project ID and stored prompt	Success
Backend API Layer	Reject invalid project request payload	HTTP `POST /api/v1/projects` with missing required fields	Response status `422` (validation error); descriptive error message returned	Success
Backend API Layer	Fetch project list when no projects exist	HTTP `GET /api/v1/projects` on a clean workspace	Response status `200`; response body is an empty list	Success
Agent Orchestration and Graph	Initialize empty pipeline state	Call state constructor in `graph/state.py` with a sample prompt	State object created with prompt set and other fields initialized to default/empty values	Success
Agent Orchestration and Graph	Manager agent produces structured requirements	Invoke Manager agent with a simple prompt (mocked LLM)	Output conforms to `ManagerOutput` schema and contains non-empty requirement fields	Success
Agent Orchestration and Graph	Architect agent generates file plan	Provide Manager output to Architect agent (mocked LLM)	Output includes a file list with valid relative paths and descriptions	Success

Module	Test Scenario	Input	Expected Result	Status
Agent Orchestration and Graph	Graph orchestrator runs full pipeline with mocked agents	Execute orchestrator with all agents mocked to return fixed outputs	Final state contains populated requirements, architecture, code artifacts, and QA data without errors	Success
Agent Orchestration and Graph	Error handling in a single node	Force an exception inside one agent node	Orchestrator catches error, records it in state/logs, and terminates or retries according to policy	Success
Storage and Project Management	Create new project directory	Call function in `project_store` with a new project ID	Corresponding folder created under `PROJECTS_DIR`; path is returned and exists on disk	Success
Storage and Project Management	Build file tree for simple project	Populate a test project with a few files and call `file_store` tree builder	Returned tree structure matches directory layout and contains correct file names and types	Success
Storage and Project Management	Read file contents safely	Request contents of a known file via `file_store`	Function returns exact file text; attempts to escape project root are rejected	Success
Storage and Project Management	Update file content	Call update function with a valid file path and new code content	File on disk is overwritten with new content; subsequent read returns updated text	Success
Storage and Project Management	Handle non-existent file read	Request a file that does not exist in the project tree	Function raises a controlled error or returns a standardized "not found" response	Success
Frontend User Interface	Render project list page with data	Mock API client to return a list of projects and render `projects/page.tsx`	Page displays the correct number of project entries with expected titles and IDs	Success
Frontend User Interface	Display file contents in code viewer	Provide a selected file and mocked content to `CodeViewer.tsx`	Component renders the content in the editor area without runtime errors	Success
Frontend User Interface	Update selected file state on tree click	Simulate click on a file node in `FileExplorer.tsx`	Global/store state updates to mark that file as selected; dependent components receive the update	Success

Module	Test Scenario	Input	Expected Result	Status
Frontend User Interface	Show execution timeline events	Supply a sequence of mocked pipeline events to `ExecutionTimeline.tsx`	Timeline renders entries in correct order with appropriate status indicators	Success
Frontend User Interface	Send chat message to backend	Trigger a send action in `ChatPanel.tsx` with mocked API client	API client is called with correct project ID and message payload; UI input is cleared	Success
LLM, SOP, and Retrieval	Construct base LLM instance	Call `get_llm()` from `llm/gemini.py` with valid configuration	Function returns a non-`None` LangChain model instance using configured parameters	Success
LLM, SOP, and Retrieval	Create structured-output LLM	Call `get_llm_with_structured_output()` with a schema	Returned model enforces the specified schema and raises an error for invalid outputs in tests	Success
LLM, SOP, and Retrieval	Load SOP definitions	Import and access SOPs from `sop/definitions.py`	Each defined agent SOP contains non-empty role, objective, inputs, outputs, and checklist fields	Success
LLM, SOP, and Retrieval	Generate embeddings for sample text	Call embedding function in `rag/embeddings.py` with a short string	Function returns a numeric vector of expected dimension without errors	Success
LLM, SOP, and Retrieval	Retrieve nearest neighbor from vector store	Index two or more documents and query via `rag/retriever.py`	Retriever returns the most similar document to the query text according to test setup	Success

5.3.2 Integration Testing

Integration testing verifies that the various subsystems of MetaGPT—FastAPI endpoints, agent pipeline, storage, and RAG utilities—work together correctly when exercised through realistic workflows. Tables 5.2–5.5 summarize integration test cases for each area, using the same template as the unit testing table.

API-level Tests

Using `pytest` and FastAPI's `TestClient`, integration tests call endpoints defined in `backend/app/api/endpoints`. Table 5.2 lists representative scenarios.

Table 5.2: Integration Test Cases: API-level Tests

Module	**Test Scenario**	**Input**	**Expected Result**	**Status**
API (Projects)	Create project and verify persistence	POST `/api/v1/projects` with `{"prompt": "Todo app"}`	Status 201; response has project `id`; project dir exists under `PROJECTS_DIR`	Success
API (Projects)	List projects after creating one	GET `/api/v1/projects` after creating a project	Status 200; list contains at least one project with matching `id` and prompt	Success
API (Pipeline)	Start pipeline run and get status	POST `/api/v1/pipeline/run` with valid project `id`; then GET `/api/v1/pipeline/{id}/status`	Run starts; status endpoint returns run state (e.g., running/completed) and artifact keys	Success
API (Pipeline)	Stream pipeline events	POST `/api/v1/pipeline/stream` with prompt; consume SSE stream	Stream yields events (e.g., manager_done, architect_done); connection stays open until completion	Success
API (Files)	Get file tree for generated project	GET `/api/v1/files/{id}/tree` for a project with generated files	Status 200; JSON tree matches on-disk structure; nodes have `name` and `type` (file/dir)	Success
API (Files)	Read file content via API	GET `/api/v1/files/{id}/content/{path}` for a known file	Status 200; body contains exact file content; content-type appropriate for text	Success
API (Chat)	Send chat message and fetch history	POST `/api/v1/chat/{id}` with message; GET `/api/v1/chat/{id}/history`	Message stored; history returns list including the new message with correct `role` and `content`	Success

Pipeline Orchestration

Tests exercise the full agent pipeline (Manager → Architect → Engineer → QA) via the LangGraph orchestrator. Table 5.3 lists key scenarios.

Table 5.3: Integration Test Cases: Pipeline Orchestration

Module	**Test Scenario**	**Input**	**Expected Result**	**Status**
Pipeline (Graph)	Full pipeline run with mocked LLM	Single prompt; all agents use mocked LLM returning valid structured outputs	Final state has requirements, architecture, file list, generated code, and QA output; no unhandled exceptions	Success
Pipeline (Graph)	State passed correctly between nodes	Inspect state after each node (Manager, Architect, Engineer, QA)	Each node receives correct input from previous node; state fields updated in order (requirements → design → code → qa)	Success
Pipeline (Services)	Pipeline service starts run and returns run ID	Call `pipeline_service` start with project ID and prompt	Returns run identifier; orchestrator invoked; state persisted or streamed as configured	Success
Pipeline (Services)	Retrieve artifacts after run	GET artifacts for a completed pipeline run ID	Response includes manager output, architect output, engineer outputs (files), QA output in expected structure	Success
Pipeline (Graph)	Error in one agent does not corrupt state	Force one agent to raise; capture state/logs	Error caught; state or logs indicate which node failed; no partial writes leave inconsistent state	Success

Storage + API Integration

End-to-end tests combine storage and API: create project, run pipeline (or seed files), then access via `/api/v1/files`. Table 5.4 summarizes test cases.

Table 5.4: Integration Test Cases: Storage + API Integration

Module	**Test Scenario**	**Input**	**Expected Result**	**Status**
Storage + API	Create project then list via API	Create project via API; add a file to project dir (simulate pipeline); GET `/api/v1/files/{id}/tree`	Tree includes the new file; path and name match filesystem	Success

Module	Test Scenario	Input	Expected Result	Status
Storage + API	Read file written by pipeline	Run pipeline (or seed file); GET `/api/v1/files/{id}/content/{path}`	Content returned matches content written by pipeline or test	Success
Storage + API	Update file via API and read back	PUT `/api/v1/files/{id}/content/{path}` with new body; GET same path	File on disk updated; GET returns new content	Success
Storage + API	Invalid path returns error	GET `/api/v1/files/{id}/content/{path}/../etc/passwd` or path outside project	Status 400/403/404; no content from outside project root returned	Success
Storage + API	Empty project returns empty tree	GET `/api/v1/files/{id}/tree` for newly created project with no files	Status 200; tree is empty or has only root node	Success

RAG Workflows

For setups where RAG is enabled, integration tests index documents and call retrieval endpoints. Table 5.5 lists representative cases.

Table 5.5: Integration Test Cases: RAG Workflows

Module	Test Scenario	Input	Expected Result	Status
RAG (Indexer)	Index small document set	Call indexer with a few code snippets or file paths	Index completes; vector store contains expected number of entries; no errors	Success
RAG (Retriever)	Query returns relevant chunks	Index 2–3 documents; query with text similar to one document	Retriever returns chunks; top result matches or is highly similar to the queried document	Success

Module	Test Scenario	Input	Expected Result	Status
RAG + API	Retrieval endpoint returns results	GET/POST RAG endpoint with project ID and query string	Status 200; response includes retrieved snippets or IDs usable for agent context	Success
RAG (Embeddings)	Embeddings consistent for same text	Call embedding function twice with same string	Both calls return identical (or numerically equal) vector	Success
RAG (Vector store)	Add and query in same session	Add documents to vector store; run query	Query returns added documents when query text is relevant; no cross-session persistence errors	Success

Integration of Backend to Frontend

Tests verify that the Next.js frontend correctly communicates with the FastAPI backend: API base URL, request/response handling, and real-time updates. Table 5.6 lists representative scenarios.

Table 5.6: Integration Test Cases: Backend to Frontend

Module	Test Scenario	Input	Expected Result	Status
Frontend + API	Projects list loads from backend	Open `/projects`; frontend calls GET `/api/v1/projects`	Page displays project list; data matches backend response; no CORS or network errors	Success
Frontend + API	Create project from UI and see in list	Submit prompt on landing/workspace; frontend POSTs to `/api/v1/projects` or pipeline	New project appears in list or workspace; project ID and prompt reflected in UI	Success
Frontend + API	File tree and content from backend	Select project; frontend GET `/api/v1/files/{id}/tree` and content endpoints	FileExplorer shows tree; CodeViewer shows file content; data matches backend storage	Success
Frontend + API	Pipeline run and streaming updates	Start pipeline from UI; frontend uses `/api/v1/pipeline/stream` or status poll	Execution timeline updates; agent outputs appear; run completes or shows progress without timeout	Success

Module	Test Scenario	Input	Expected Result	Status
Frontend + API	Chat message sent and history loaded	Send message in ChatPanel; frontend POST `/api/v1/chat/{id}`; fetch history	Message appears in UI; history GET returns same message; no duplicate or lost messages	Success
Frontend + API	API base URL and env configuration	Run frontend with `NEXT_PUBLIC_API_URL` pointing to backend	All API calls target correct host/port; no mixed content or wrong-origin errors	Success

These integration tests are run alongside unit tests with:

```
cd backend
uv run pytest
```

and are often marked or grouped to distinguish slower, end-to-end scenarios from fast unit tests. For the frontend, integration is validated primarily by running the application against a local backend, while automated checks such as `npm run lint` and `npm run type-check` ensure consistency between API usage and TypeScript types.

5.3.3 System Testing

System testing validates the MetaGPT project as a whole from an end-user perspective: full workflow from prompt entry to generated project, UI responsiveness, and correctness of end-to-end behaviour. Table 5.7 summarizes system-level test cases.

Table 5.7: System Test Cases: Project as a Whole

Module	Test Scenario	Input	Expected Result	Status
End-to-end	Create project from prompt to codebase	User enters prompt (e.g., "Build a todo app with React"); runs pipeline; waits for completion	Project created; pipeline runs Manager → Architect → Engineer → QA; generated files appear in file tree; no fatal errors	Success
End-to-end	View and browse generated project	User opens project workspace after pipeline completion	File tree shows generated structure; selecting a file displays content in code viewer; paths and content match backend	Success

Module	Test Scenario	Input	Expected Result	Status
End-to-end	Iterate via chat and re-run pipeline	User sends chat message requesting a change; triggers follow-up run or refinement	Chat history updated; pipeline or agent responds; project state remains consistent; UI reflects changes	Success
End-to-end	Multiple projects in sequence	User creates Project A, then Project B; lists projects; opens each	Both projects listed; switching between projects shows correct file tree and content for each; no data mixed between projects	Success
End-to-end	Execution timeline reflects pipeline progress	User starts pipeline; observes timeline during run	Timeline shows agent steps (Manager, Architect, Engineer, QA) in order; status indicators update; completion state shown	Success
End-to-end	Landing and navigation	User opens app; navigates Landing → Projects → Project workspace	All pages load; navigation works; no blank screens or console errors; backend reachable from frontend	Success
End-to-end	Error handling when backend unavailable	Stop backend; perform action from frontend (e.g., load projects)	Frontend shows error or fallback; no uncaught exception; user can retry or see clear message	Success
End-to-end	Generated project structure is usable	Inspect generated project on disk (e.g., `PROJECTS_DIR`)	Directory structure exists; key files (e.g., package.json, main components) present; structure aligns with Architect output	Success

5.4 Experimental Results & Discussion

To evaluate MetaGPT, a series of experiments were performed using representative prompts such as "Build a todo app with React and a REST API backend" and "Build a blog platform with Next.js". For each prompt, the agent pipeline was executed end-to-end, and the resulting project structure, code quality, and test coverage were examined. The system successfully generated multi-file codebases with clear separation between frontend and backend components, along with auxiliary files such as configuration and routing modules.

Qualitative analysis showed that the SOP-driven pipeline led to more consistent architectures and clearer traceability from requirements to implementation compared to ad-hoc single-prompt generation. The execution timeline and agent output visualization also improved transparency, allowing users to understand and debug the reasoning process of each agent. Limitations observed during experimentation include sensitivity to ambiguous prompts and occasional need for manual adjustments to align generated code with specific framework versions or organizational coding standards. These observations inform the future enhancements discussed in later chapters.

Chapter 6

Conclusion

This project presented the design and implementation of MetaGPT, a production-ready agentic system for transforming natural language prompts into complete software projects. By modeling the roles of a Manager, Architect, Engineer, and QA agent, and by defining clear SOPs for each, the system brings structure and traceability to LLM-based code generation. The FastAPI backend, LangGraph-based pipeline, and Next.js frontend together provide an integrated environment where users can create projects, observe pipeline execution, inspect generated artifacts, and iteratively refine outputs through chat.

Through implementation and testing, MetaGPT demonstrated that an SOP-driven, multi-agent approach can produce more consistent architectures and clearer reasoning trails than one-shot code generation tools. The system highlights the importance of modular design, explicit state management, and rich visualization when building LLM-powered developer tools. While some limitations remain, particularly around handling highly ambiguous requirements and domain-specific constraints, the work establishes a solid foundation for future research and practical deployment of agentic software engineering systems.

Chapter 7

Future Scope

MetaGPT opens several avenues for future enhancement and research. While the system already supports a range of project types—including web applications, APIs, CLIs, and libraries—one promising direction is improving the reliability and live-preview support for non-web project types such as mobile applications, microservices, and data engineering pipelines. The current E2B sandbox preview is optimised for frontend web projects; extending scaffolding logic and sandbox templates to cover additional frameworks and deployment targets would broaden the system's practical utility.

Another important area is deeper integration with developer workflows, including Git-based version control and continuous integration (CI) pipelines. Features such as automatic commit generation and pull request creation would further reduce friction between generated projects and real-world engineering practices. Additionally, incorporating richer feedback loops—for example, using runtime metrics, static analysis tools, or human-in-the-loop annotations—could help the agents iteratively improve code quality and performance over multiple pipeline runs.

From a research perspective, MetaGPT can serve as a testbed for studying collaboration between multiple LLM agents, long-lived project memory across sessions, and adaptive SOPs that evolve over time. The system currently uses Google Gemini exclusively via LangChain; experimenting with alternative LLM backends and multi-model setups, as well as hybrid workflows where human experts supervise or override specific agent stages, are also valuable directions for future work.

References

[1] H. Hong, Y. Du, J. Pan, Z. Wu, and W. Chen, "MetaGPT: Meta Programming for Multi-Agent Collaborative Framework," *arXiv preprint arXiv:2308.00352*, 2023.

[2] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, S. Chan, A. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. Such, K. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, A. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating Large Language Models Trained on Code," *arXiv preprint arXiv:2107.03374*, 2021.

[3] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. D. Lago, J. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P. S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. S. G. Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, "Competition-Level Code Generation with AlphaCode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[4] H. Yang, S. Yue, and Y. He, "Auto-GPT for Online Decision Making: Benchmarking and Improving," *arXiv preprint arXiv:2304.12411*, 2023.

[5] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. D. Lago, J. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P. S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. S. G. Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals, "MapCoder: Multi-Agent Code Generation for Competitive Problem Solving," *arXiv preprint arXiv:2401.14165*, 2024.

[6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv preprint arXiv:2210.03629*, 2022.

[7] N. Shinn, F. Labash, and Y. Goksel, "Reflexion: An Autonomous Agent with Dynamic Memory and Self-Reflection," *arXiv preprint arXiv:2303.11366*, 2023.

[8] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," *arXiv preprint arXiv:2302.04761*, 2023.

[9] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, J. Xu, Z. Liu, and M. Sun, "ChatDev: Communicative Agents for Software Development," *arXiv preprint arXiv:2307.07924*, 2023.

[10] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is All You Need," *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.

[12] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.

[13] LangChain, "LangChain Documentation," Available: `https://python.langchain.com/`, Accessed: Oct. 2025.

[14] LangGraph, "LangGraph Documentation," Available: `https://langchain-ai.github.io/langgraph/`, Accessed: Oct. 2025.

[15] Google, "Gemini API Documentation," Available: `https://ai.google.dev/docs`, Accessed: Oct. 2025.

[16] Tree-sitter, "Tree-sitter Documentation," Available: `https://tree-sitter.github.io/`, Accessed: Oct. 2025.

Appendix

Appendix I – Code Snippets

This appendix can include representative code snippets from the MetaGPT project, such as:

- The FastAPI application entry point that wires routers and middleware.
- An example of an agent SOP definition or agent implementation.
- A LangGraph pipeline definition showing the connections between Manager, Architect, Engineer, and QA agents.
- A sample frontend component (e.g., `ExecutionTimeline` or `FileExplorer`) illustrating how agent outputs are visualized.

These snippets help readers understand the implementation style, coding conventions, and integration points between different modules of the system.

List of Publications

Your publication details here.

Achievements If Any

Your achievements here.